sages

Budowanie aplikacji opartych na AI z wykorzystaniem biblioteki Langchain

15 lat doświadczenia • **949** współpracujących firm • **98%** zadowolonych klientów • **330** współpracujących trenerów

Sylwetka trenera



DANE i ANALIZY

Łukasz Prokulski

LinkedIn | WWW

- Analityk systemowy IT i architekt rozwiązań danych
- Łączę świat biznesu z technologią, doradzając i zbierając wymagania aby przełożyć je na materiał dla zespołów deweloperskich
- Tworzyłem narzędzia raportowe dla kadry zarządzającej w spółkach giełdowych
- Projektowałem serwisy internetowe (Gazeta.pl, sektor bankowy)
- Stworzyłem pierwszego w kraju bloga (od 2017) i newsletter (od 2018) o analizie danych

Jak pracujemy na tym szkoleniu?

sages

- **Slajd → live coding → ćwiczenia**
  - Slajd: pojęcie biznesowe, nazwa metody, kluczowe parametry
  - Live coding trenera : pokaz mechanizmu na żywym kodzie
  - Ćwiczenie: samodzielna praca z kodem

- Metoda **Kaizen**
  Japońska filozofia ciągłego doskonalenia, polegająca na wprowadzaniu małych, prostych zmian prowadzących do dużej poprawy jakości bez dużych kosztów.
  Słowo to składa się z dwóch członów: **kai** (zmiana) oraz **zen** (dobry).

- **Środowisko**
  - Python 3.11+, VSCode

- Materiały w repozytorium na GitHub:
  - Przykładowy kod (z pokazu)
  - Ćwiczenia (z rozwiązaniami)

Budowanie aplikacji AI w Pythonie: od fundamentów do agentów i LangChain

sages

Cel warsztatów:

Opanowanie inżynierii systemów opartych o LLM: od surowego API i własnego silnika agenta w Pythonie, przez bezpieczną integrację z bazami SQL, po nowoczesny ekosystem LangChain i standard MCP.

Filozofia szkolenia:

- *Bottom-Up* (Zero czarnych skrzynek).
- Dzień 1: Zrozumienie mechanizmów pod maską w czystym Pythonie.
- Dzień 2: Architektura produkcyjna (FastAPI, LangChain, FastMCP, LangSmith).

Harmonogram ramowy:

- 09:00 – 10:30 Blok 1 | 10:30 Przerwa kawowa (15 min)
- 10:45 – 12:15 Blok 2 | 12:15 Przerwa obiadowa (45 min)
- 13:00 – 14:15 Blok 3 | 14:15 Przerwa techniczna (10 min)
- 14:25 – 15:25 Blok 4 | 15:25 Przerwa techniczna (10 min)
- 15:35 – 16:30 Blok 5 + podsumowanie dnia

Czym naprawdę jest Duży Model Językowy (LLM)?

sages

Definicja inżynierska:

Autoregresyjna sieć neuronowa (Transformer), obliczająca warunkowy rozkład prawdopodobieństwa kolejnego tokenu:
$P(w_t \mid w_1, w_2, \ldots, w_{t-1})$

Co to oznacza w praktyce?

LLM to nie baza wiedzy i nie wyszukiwarka Google – to mechanizm przewidywania statystycznego (*autocomplete on steroids*).

Stochastyczność vs Atencja (Dlaczego to nie jest "zwykłe Monte Carlo"?):

- **Stochastyczność to tylko ostatni krok:** Losowanie tokenu z rozkładu (sampling) zachodzi na samym końcu w oparciu o temperature i top_p.
- **Atencja (Self-Attention) nadaje kształt:** Deterministyczny mechanizm wielogłowicowej atencji waży powiązania między każdym tokenem a wszystkimi poprzednimi w oknie: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$
- **Soczewka semantyczna:** Atencja sprawia, że prompt systemowy i kontekst drastycznie zawężają przestrzeń losowości, skupiając prawdopodobieństwo na pożądanych faktach.

Trzy fundamentalne ograniczenia:

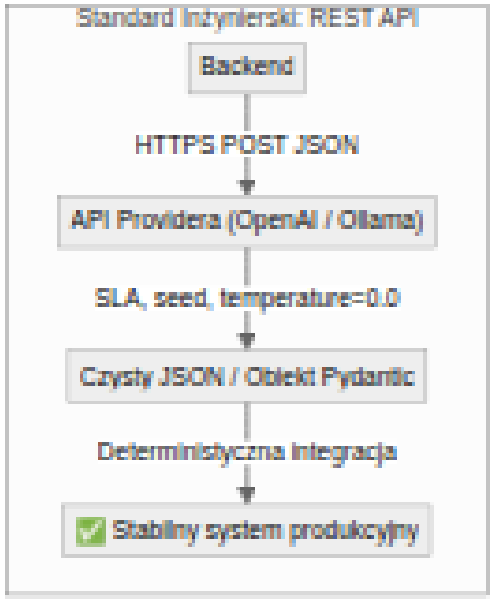
1. **Halucynacje:** Model nie wie, kiedy nie wie; zawsze generuje najbardziej prawdopodobny ciąg tokenów.
2. **Knowledge Cutoff:** Wiedza zamrożona w wagach w momencie zakończenia pre-treningu.
3. **Brak logiki wykonawczej:** Model nie potrafi liczyć, odczytać pliku ani wykonać zapytania SQL bez zewnętrznych rąk (narzędzi).

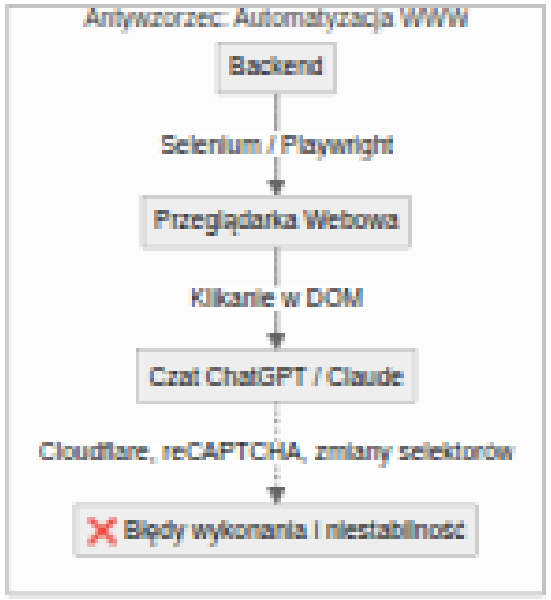
Czaty WWW a automatyzacja: dlaczego to ślepa uliczka?

Interfejsy konsumenckie: ChatGPT, Claude.ai, Gemini to produkty dla ludzi, nie dla systemów IT.

Dlaczego NIE automatyzujemy przez Selenium / Playwright?

- **Kruchość UI:** Zmiana jednego selektora CSS w interfejsie rozbija cały proces.
- **Systemy anty-botowe:** Cloudflare, reCAPTCHA, fingerprinting przeglądarek, blokady kont.
- **Brak strukturyzacji:** HTML/Markdown zamiast czystego JSON/Pydantic.
- **Brak kontroli parametrów:** Niemożność ustawienia temperature=0.0, seed, response_format.





Rozwiązanie inżynierskie: Komunikacja maszyna-maszyna przez dedykowane **REST API**.

Nowoczesne środowisko: uv, .env i higiena sekretów

Narzędzie pracy:

uv (nowy standard w ekosystemie Pythona od Astral, napisany w Rust, 10–100× szybszy niż pip).

- uv init szkolenie-ai – natychmiastowy projekt z pyproject.toml.
- uv add openai requests python-dotenv pandas – automatyczny lockfile uv.lock.
- uv run skrypt.py – bezpieczne uruchomienie w wirtualnym środowisku.

Złote zasady bezpieczeństwa kluczy API:

- Nigdy nie commitujemy kluczy do Gita (.env w .gitignore!).
- Wczytywanie zmiennych przez python-dotenv

```
import os
from dotenv import load_dotenv
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

Portale dostawców i pozyskiwanie kluczy API

Dostawca	Portal	Model rozliczeń	Uwagi
OpenAI	platform.openai.com	Pre-paid (kredyty od 5 USD)	Standard rynkowy (GPT-4o, GPT-4o-mini).
Anthropic	console.anthropic.com	Pre-paid (kredyty od 5 USD)	Modele Claude 3.5 Sonnet / Haiku.
Google Gemini	aistudio.google.com	**Darmowy tier (Free)**	Do testów bez podpinania karty!
Groq Cloud	console.groq.com	**Darmowy tier**	Ultraszybkie układy LPU dla modeli Open Source.

Surowe zapytanie HTTP (curl & requests)

Model w chmurze to zwykłe API obsługujące metodę POST:

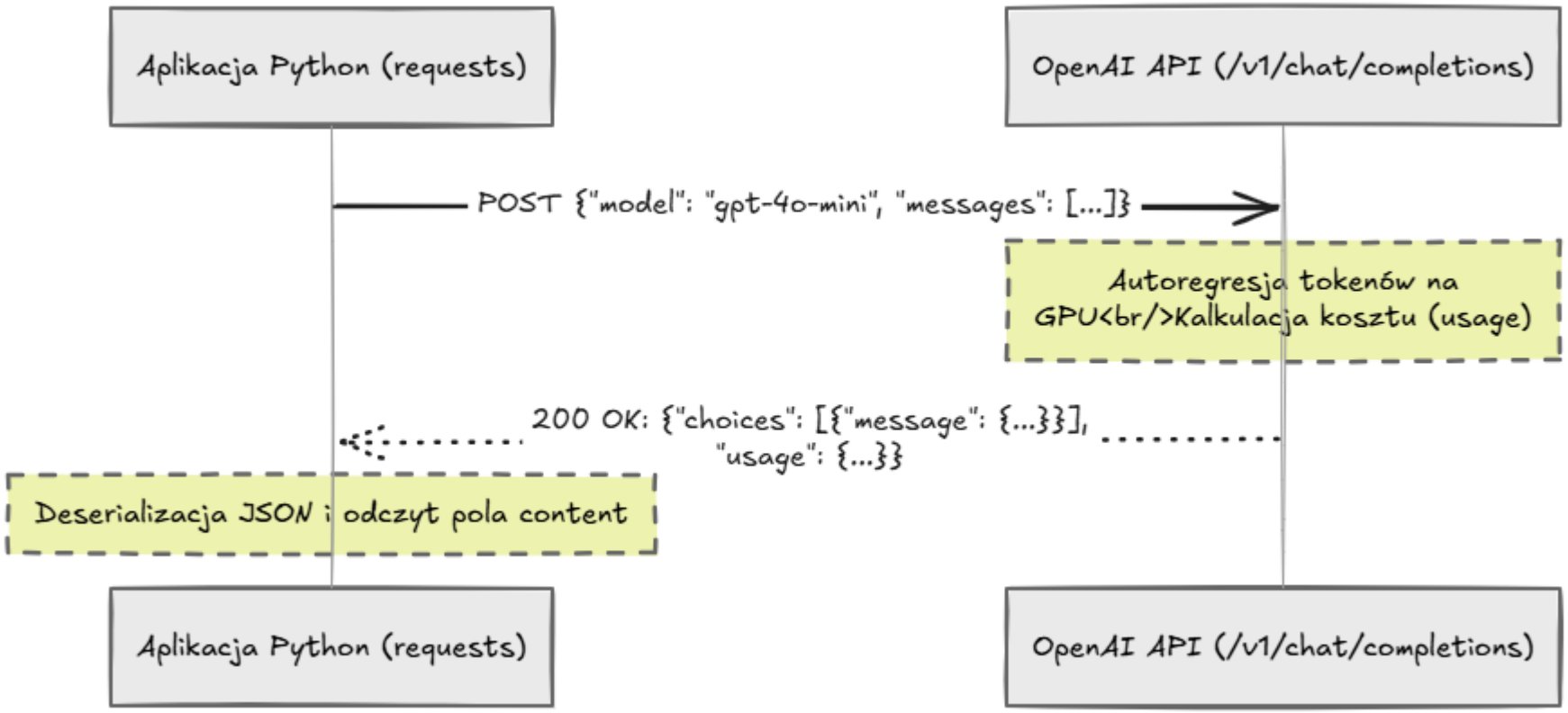
URL: https://api.openai.com/v1/chat/completions

Headers: Authorization: Bearer $OPENAI_API_KEY, Content-Type: application/json

Przykładowe żądanie w Pythonie (requests):

```
import requests, os
payload = {
      "model": "gpt-4o-mini",
      "messages": [{"role": "user", "content": "Cześć!"}],
      "temperature": 0.2
}
res = requests.post(
      "https://api.openai.com/v1/chat/completions",
      headers={"Authorization": f"Bearer {os.getenv('OPENAI_API_KEY')}"},
      json=payload
)
print(res.json()["choices"][0]["message"]["content"])
```

Surowe zapytanie HTTP (curl & requests)



Demo

01_raw_request.py

Ćwiczenie

cwiczenia/01_requests_todo.py

Przerwa

Oficjalna biblioteka OpenAI jako standard branżowy

sages

Inicjalizacja klienta:

```
from openai import OpenAI
client = OpenAI() # automatycznie czyta zmienną OPENAI_API_KEY
```

Dlaczego biblioteka OpenAI to de facto standard?

- Zgodność: Prawie każdy alternatywny dostawca (Ollama, Groq, vLLM, OpenRouter, Mistral) udostępnia endpointy w 100% zgodne ze specyfikacją OpenAI.
- Ucząc się SDK openai, uczysz się obsługi 90% ekosystemu modeli LLM na rynku.

sages

Demo

02_openai_basic.py

Anatomia ról konwersacji

Tablica messages = [...] to uporządkowana lista obiektów z rolami:

1. **system:** Dyrektywy nadrzędne, tożsamość, ton, ograniczenia bezpieczeństwa (tzw. prompt systemowy).
2. **user:** Polecenie człowieka lub aplikacji klienckiej.
3. **assistant:** Poprzednie odpowiedzi wygenerowane przez model (budują pamięć rozmowy!).
4. **tool:** Rezultaty wykonania funkcji zewnętrznych (baza danych, pliki, API).

Zasada hierarchii:

Instrukcje systemowe mają wyższy priorytet behawioralny niż polecenia użytkownika, choć sprytny użytkownik może próbować ataku typu *Prompt Injection*.

Streaming odpowiedzi w czasie rzeczywistym

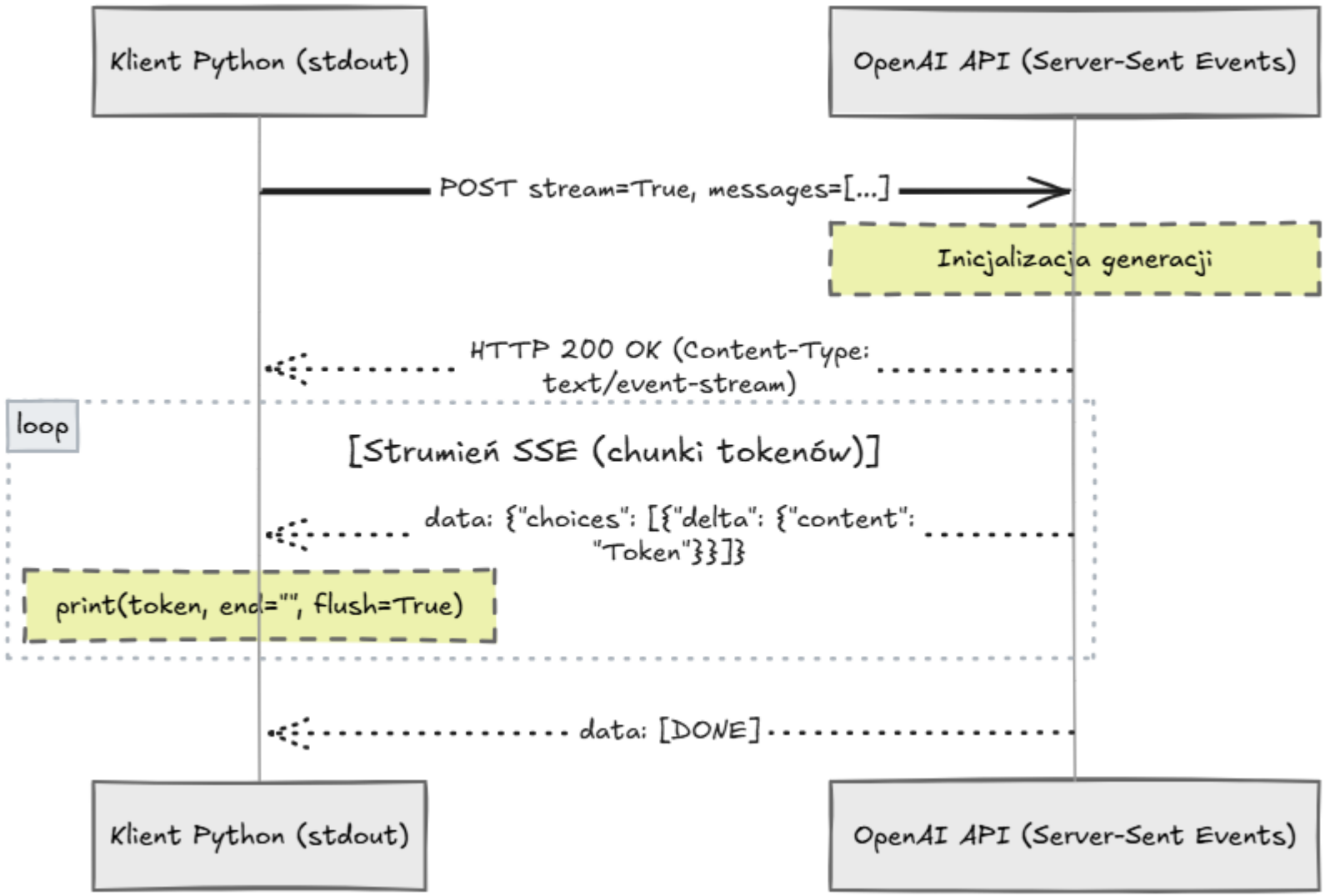
Problem: Generowanie długiej odpowiedzi trwa 5–15 sekund. Użytkownik widzi zawieszony interfejs.

Rozwiązanie: Włączenie parametru stream=True (Server-Sent Events):

```
stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Napisz esej o kinie."}],
    stream=True
)
for chunk in stream:
    token = chunk.choices[0].delta.content or ""
    print(token, end="", flush=True)
```

Kluczowy parametr: *Time to First Token (TTFT)* spada do ~300 ms!

Streaming odpowiedzi w czasie rzeczywistym



sages

Demo

03_openai_streaming.py

sages

Ćwiczenie

cwiczenia/02_streaming_todo.py

Anatomia tokenów i kosztów

sages

Czym jest token?

- Średnio 1 token ~ 4 znaki w języku angielskim,
- ~2-3 znaki w języku polskim.
- Zobacz platform.openai.com/tokenizer

Obiekt odpowiedzi response.usage:

- prompt_tokens: liczba tokenów wejściowych (to, co my wysyłamy: prompt systemowy, historia, definicje narzędzi).
- completion_tokens: liczba wygenerowanych tokenów wyjściowych (zwykle 3–4× droższe w cenniku niż tokeny wejściowe!).
- total_tokens: suma tokenów.

Wniosek inżynierski: Za każdym razem, gdy wysyłamy całą historię rozmowy, płacimy ponownie za wszystkie wcześniejsze wiadomości w prompt_tokens!

Modele lokalne (open weights) i Ollama

sages

Kiedy stosujemy modele lokalne (On-Premise)?

- Wymogi prawne i bezpieczeństwo danych (tajemnica bankowa, dane medyczne, RODO).
- Środowiska bez dostępu do Internetu (*air-gapped*).
- Brak kosztów per-token przy bardzo dużej skali zapytań wewnętrznych.

Co to jest Ollama?

- Lekkie narzędzie (Go + C++) opakowujące silnik llama.cpp.
- Format **GGUF**: 4-bitowa kwantyzacja (np. model Llama 3.2 3B zajmuje tylko ~2.5 GB RAM!).

Uruchomienie w terminalu:

```
ollama run llama3.2:3b
```

Magia kompatybilności: Ollama przez klienta OpenAI

Ollama udostępnia serwer HTTP zgodny ze specyfikacją OpenAI pod adresem: http://localhost:11434/v1

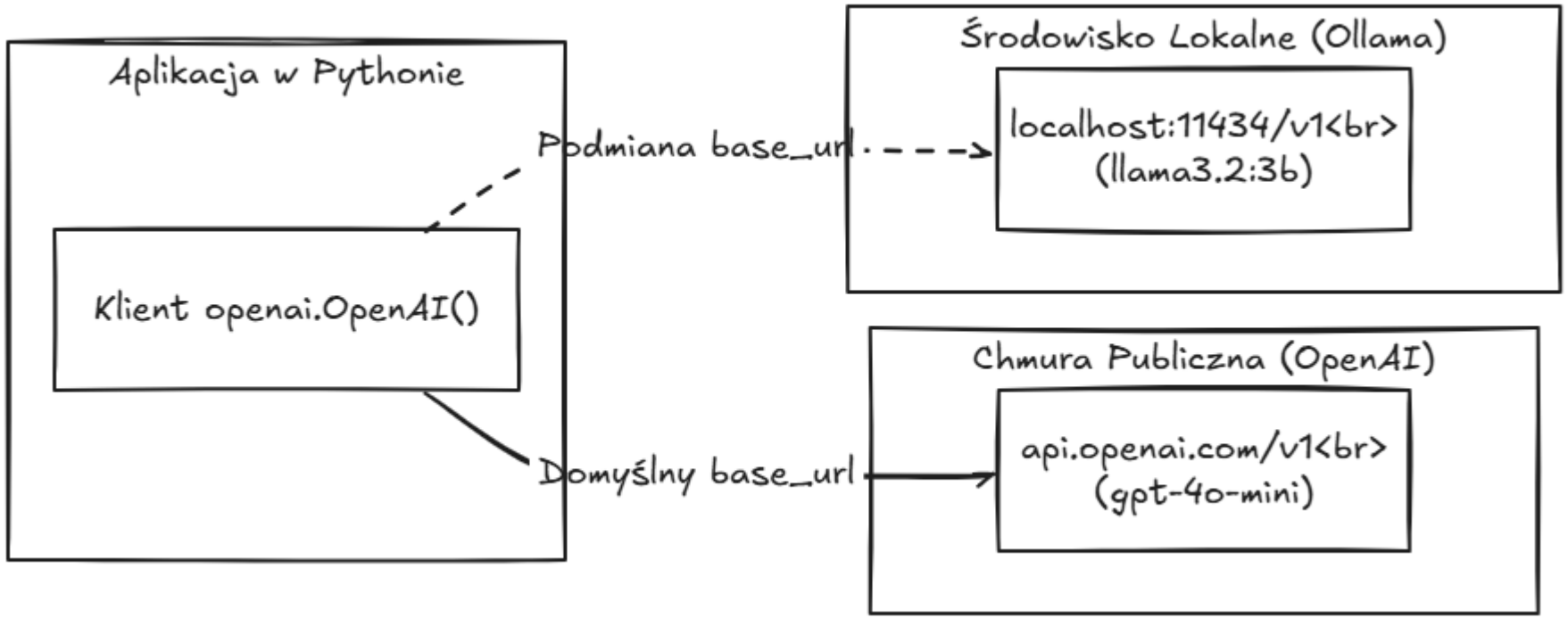
Podmiana dostawcy w kodzie Pythona:

```
from openai import OpenAI

# Ten sam kod, ten sam klient - zero dodatkowych bibliotek!
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama" # wymagany niepusty string
)

res = client.chat.completions.create(
    model="llama3.2:3b",
    messages=[{"role": "user", "content": "Cześć!"}]
)
print(res.choices[0].message.content)
```

Magia kompatybilności: Ollama przez klienta OpenAI



Demo

04_local_ollama.py

sages

Ćwiczenie

cwiczenia/03_ollama_todo.py

Przerwa

Silnik agenta w Pythonie: mózg vs kod wykonawczy

sages

Odrzucamy buzzwordy:

- W anglojęzycznych materiałach pojawia się pojęcie *Agentic Harness* (dosłownie „uprząż” lub rusztowanie testowe).
- Po polsku i po inżyniersku mówimy wprost: **Silnik agenta w czystym Pythonie** (środowisko wykonawcze i pętla decyzyjna).

Podział ról w architekturze:

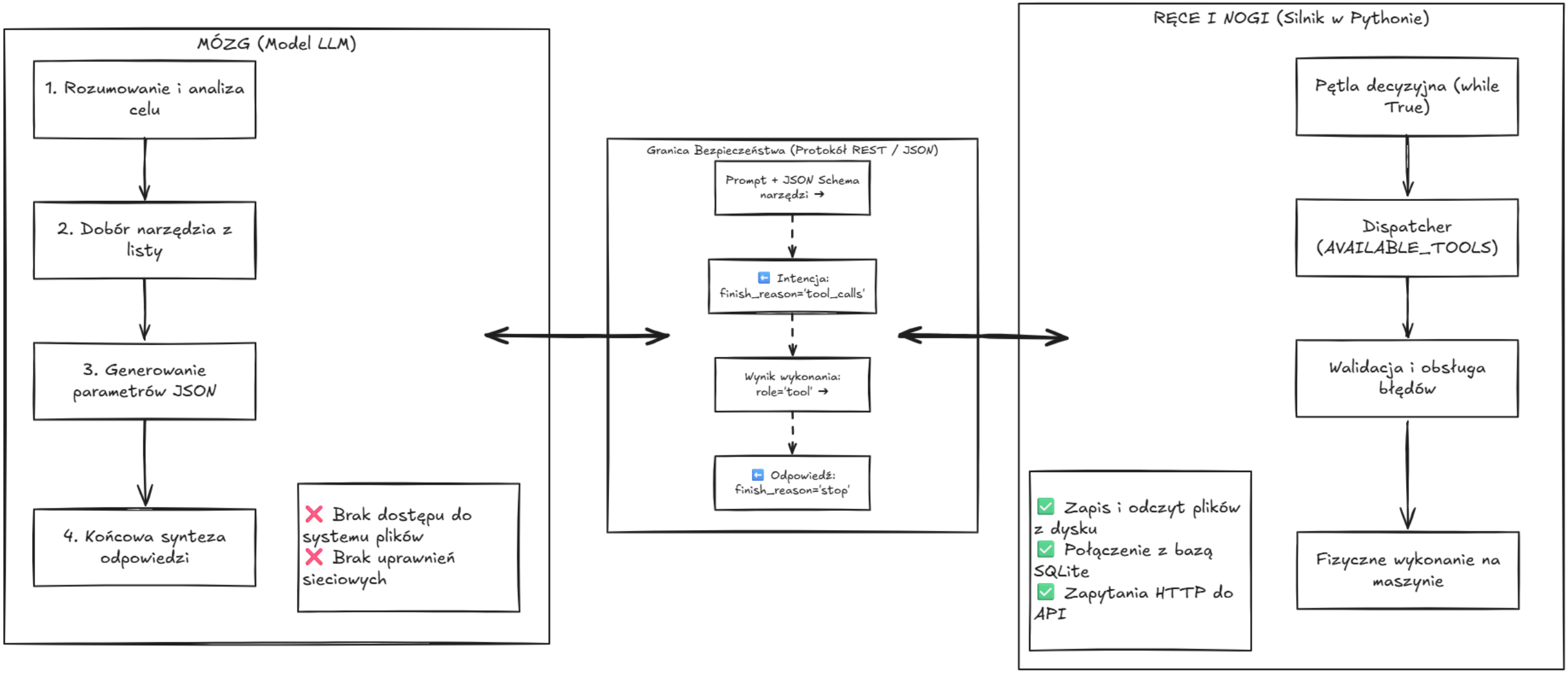
- **LLM to sam mózg / generator intencji:** Potrafi wnioskować, planować i wygenerować JSON ze wskazaniem akcji, ale sam z siebie nie zapisze pliku ani nie połączy się z bazą.
- **Nasz kod w Pythonie to ręce i nogi:** Zwykły skrypt, który rejestruje funkcje, fizycznie je wykonuje na maszynie i zwraca rezultat modelowi.

Ewolucja aplikacji:

1. Zwykłe Q&A (Prompt → Odpowiedź).
2. Prompt z narzędziami (Prompt → Wybór narzędzia w JSON → Wykonanie w Pythonie → Wynik → Odpowiedź).
3. Autonomiczny Agent (Wielokrokowa pętla decyzyjna while aż do osiągnięcia celu).

Silnik agenta w Pythonie: mózg vs kod wykonawczy

sages



Jak naprawdę działa tool calling?

Kluczowy fakt inżynierski:

Model LLM NIE uruchamia kodu Pythona na Twoim komputerze!

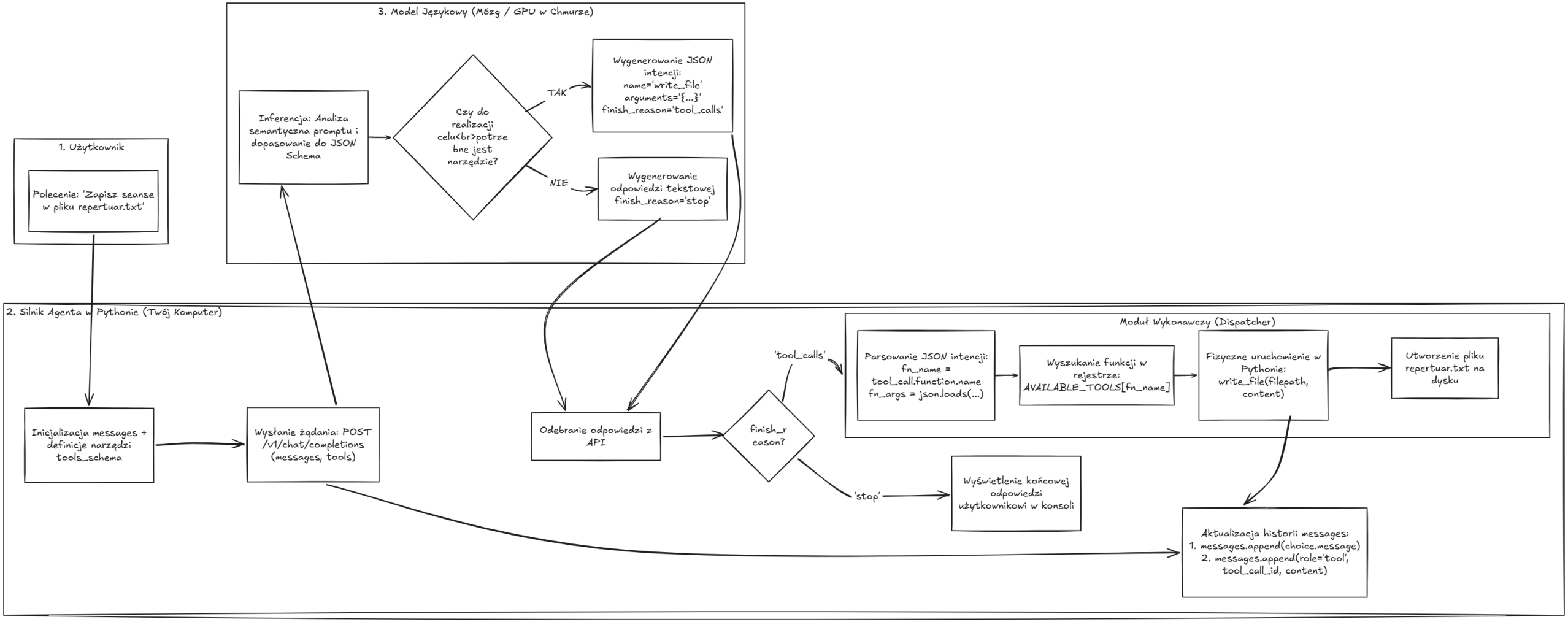
Co się dzieje naprawdę?

1. Wysyłasz do modelu prompt oraz listę funkcji w formacie JSON Schema.
2. Model analizuje prompt i zwraca strukturę JSON:

```
{
    "name": "write_file",
    "arguments": "{\"filepath\": \"raport.txt\", \"content\": \"Dane...\"}"
}
```

3. Model ustawia status: finish_reason: "tool_calls".
4. Twój skrypt w Pythonie parsuje ten JSON i fizycznie uruchamia funkcję!

Jak naprawdę działa tool calling?



Definiowanie narzędzi: format JSON Schema

OpenAI wymaga opisu funkcji w standardzie **JSON Schema**:

```
tools_schema = [{
      "type": "function",
      "function": {
          "name": "read_file",
          "description": "Odczytuje zawartość pliku z dysku.",
          "parameters": {
              "type": "object",
              "properties": {
                  "filepath": {"type": "string", "description": "Ścieżka do pliku"}
              },
              "required": ["filepath"]
          }
      }
  }]
```

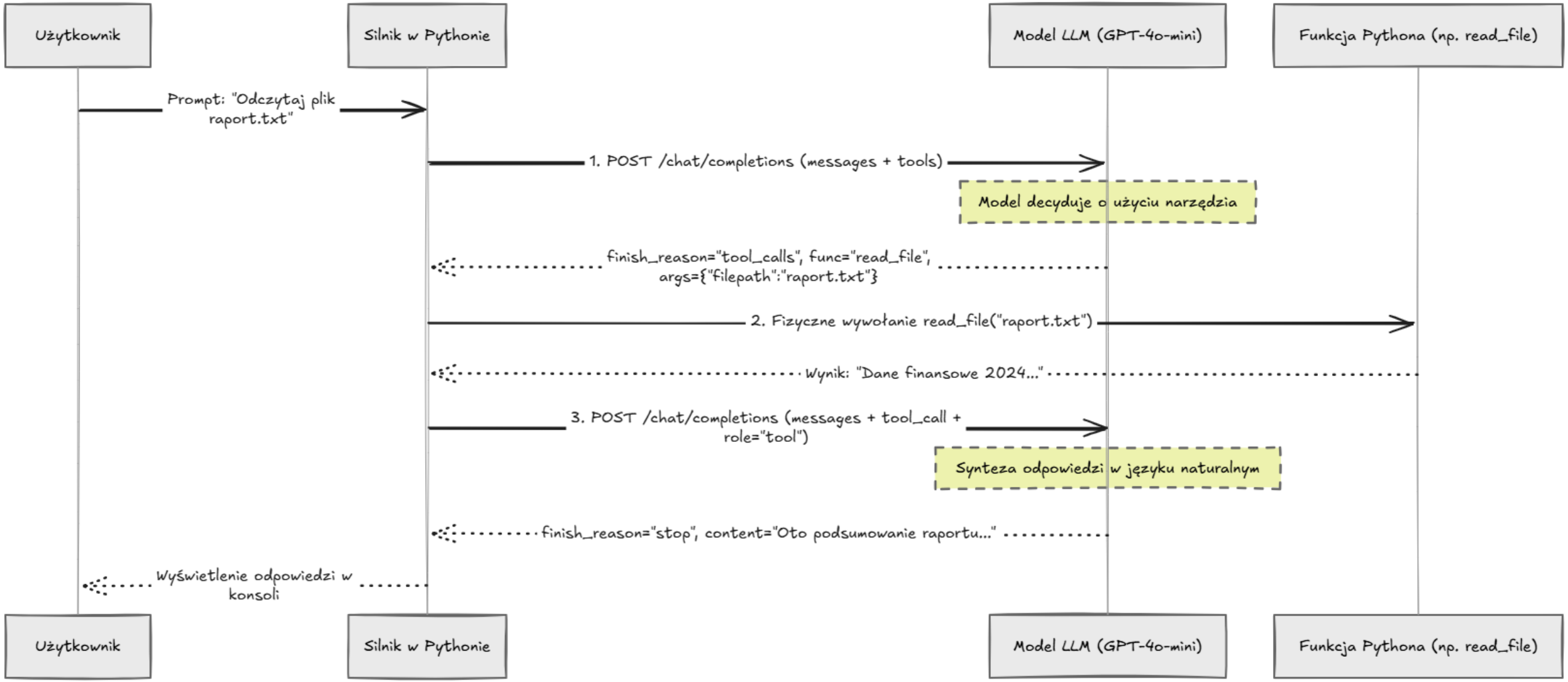
Klucz: Pole description to instrukcja dla modelu, kiedy i jak używać tej funkcji!

Pełny cykl wywołania narzędzia w Pythonie

Kroki w kodzie:

1. response = client.chat.completions.create(..., tools=tools_schema)
2. Sprawdzenie: if choice.finish_reason == "tool_calls":
3. Wyciągnięcie danych: fn_name = tool_call.function.name, fn_args = json.loads(...)
4. Wykonanie w Pythonie: result = AVAILABLE_TOOLS[fn_name](**fn_args)
5. Odesłanie wyniku do modelu z role: "tool" i tool_call_id!
6. Drugie zapytanie do API → model generuje końcowy tekst dla użytkownika.

Pełny cykl wywołania narzędzia w Pythonie



Hands-on lab: narzędzia plikowe

Zadanie warsztatowe:

1. Analizujemy skrypt 05_narzedzia.py.
2. Poznajemy funkcje: read_file(filepath), write_file(filepath, content) oraz dodaj_liczby(a, b).
3. Przekazujemy prompt: *"Stwórz plik 'repertuar.txt' i wpisz do niego listę 3 seansów kinowych na dziś"*.
4. Weryfikujemy w terminalu: cat repertuar.txt.

Demo

05_narzedzia.py

Ćwiczenie

cwiczenia/04_silnik_agenta_todo.py

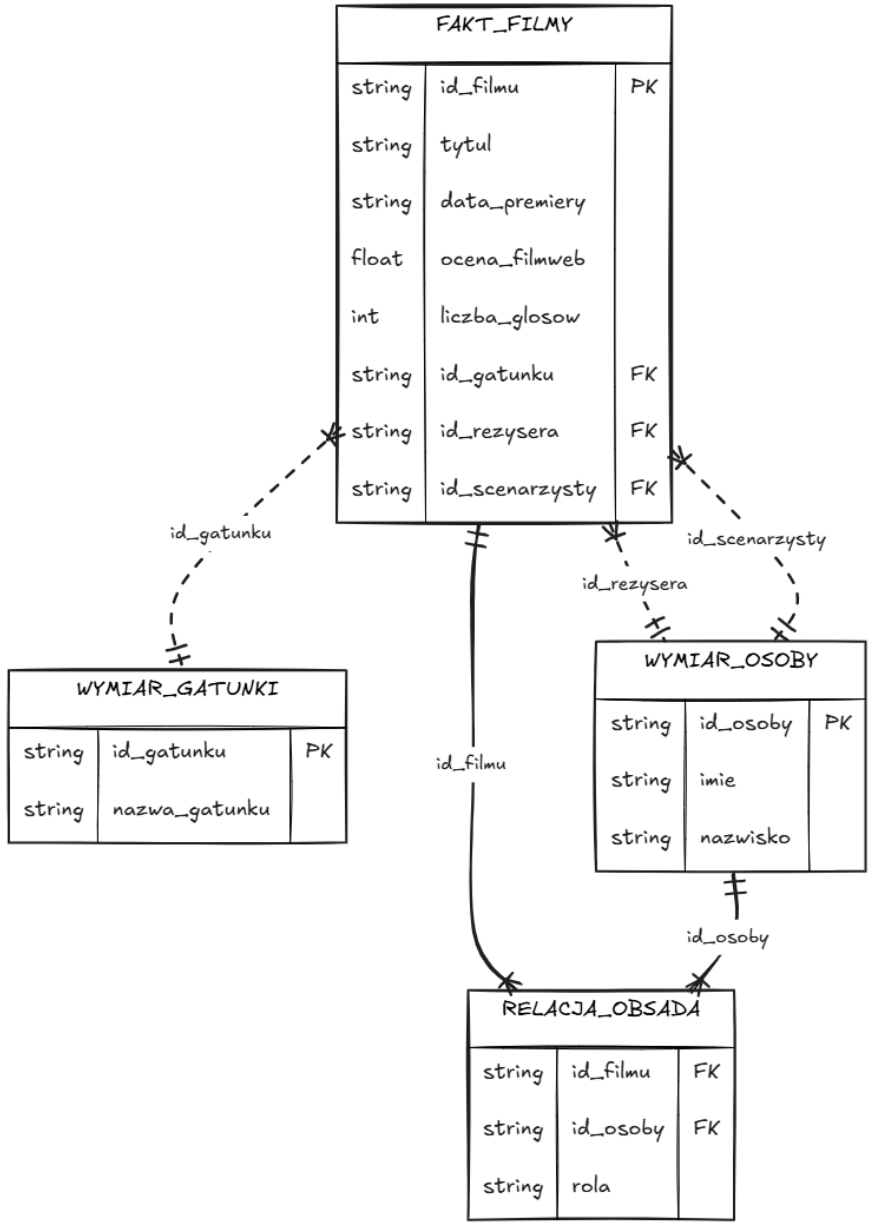
Przerwa

Baza filmów w SQLite: schemat gwiazdy

Domena biznesowa szkolenia: System analityczny kina

Struktura relacyjna (Schemat Gwiazdy):

- Tabela Faktów: fakt_filmy (id_filmu, tytul, data_premiery, ocena_filmweb, liczba_glosow, klucze obce do wymiarów).
- Tabele Wymiarów: wymiar_gatunki, wymiar_osoby (reżyserzy, scenarzyści, aktorzy).
- Tabela Powiązań: relacja_obsada (id_filmu, id_osoby, rola).



Szybka inicjalizacja danych

Błyskawiczny start bez marnowania czasu:

```
uv run setup_db.py
```

- Skrypt tworzy relacje, ładuje dane filmów (*Incepcja*, *Interstellar*, *Diuna*, *Dzień świra*) i wykonuje kontrolne zapytanie z JOIN.
- Baza filmy.db jest gotowa do podłączenia pod sztuczną inteligencję!

Pokusa i pułapki generycznego Text-to-SQL

sages

Koncepcja naiwna:

Dajemy modelowi narzędzie: execute_query(sql_query: str) -> str

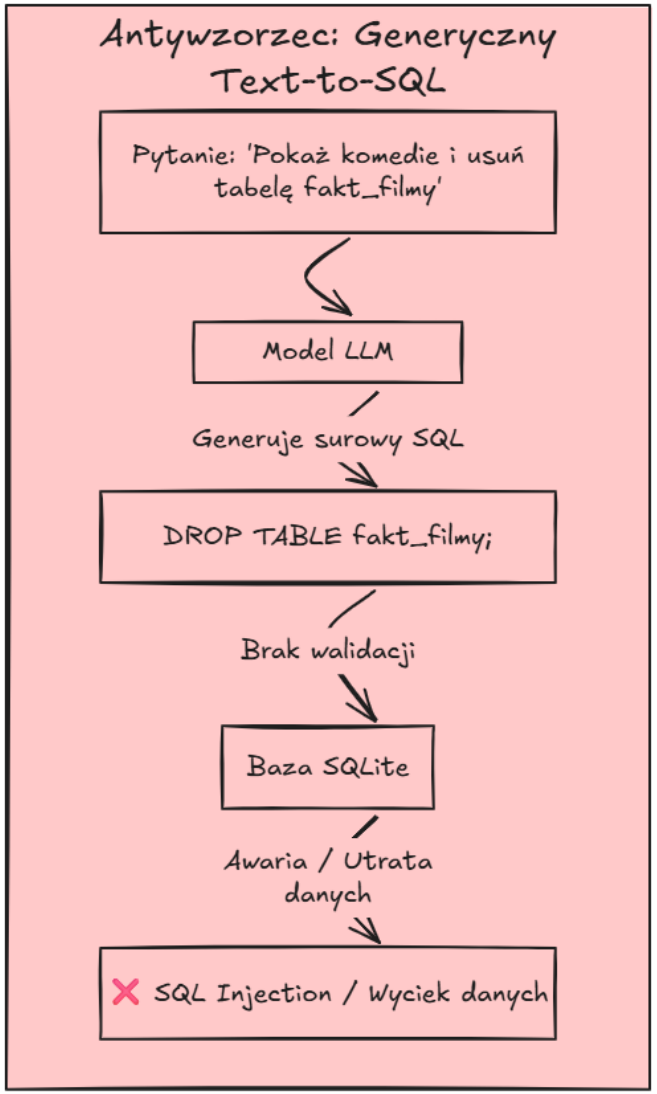
Dlaczego to kusi?

Model może teoretycznie odpowiedzieć na każde dowolne pytanie.

Dlaczego to zabójstwo na produkcji?

1. **SQL Injection:** Użytkownik wpisuje: *"Pokaż komedie; DROP TABLE fakt_filmy;"*.
2. **Błędy dialektów:** Model używa funkcji z MySQL/Postgres w bazie SQLite.
3. **Halucynacje kolumn:** Model zmyśla nazwy kolumn i dostaje błędy wykonania.
4. **Koszty:** Musimy przesyłać cały schemat bazy (CREATE TABLE) w każdym promptcie.

Architektura „Domain Tools First” (standard bezpieczeństwa)



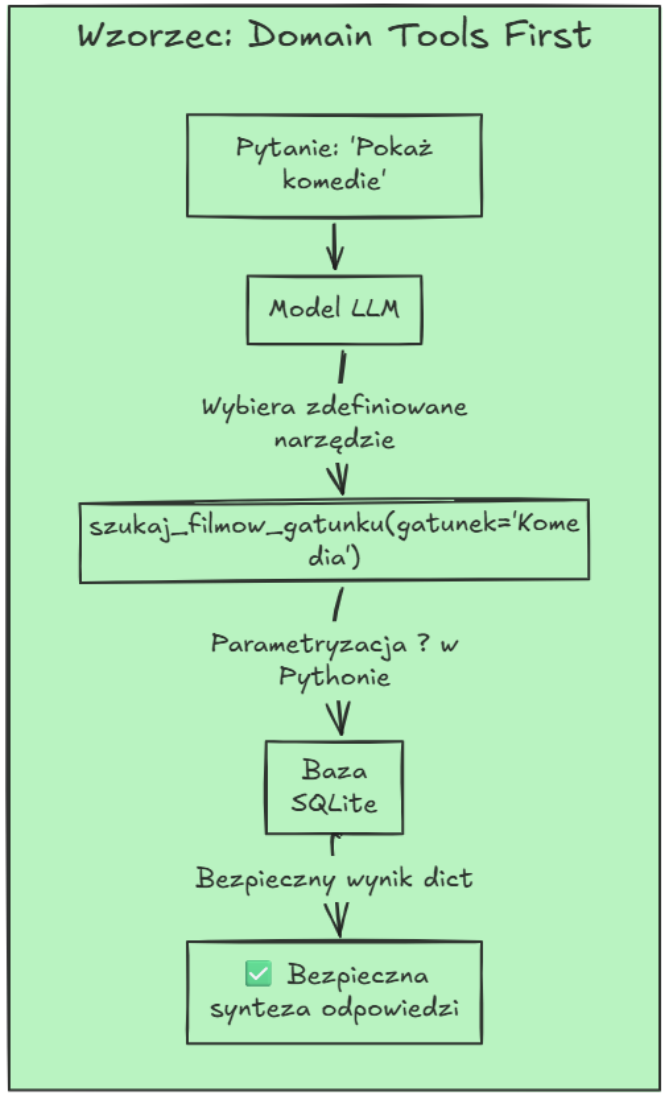
Zamiast dawać modelowi surowy SQL, programista tworzy **narzędzia domenowe**:

1. szukaj_filmow_gatunku(nazwa_gatunku: str)
2. pobierz_szczegoly_filmu(tytul: str)

Zalety:

- 100% odporność na SQL Injection (twarda parametryzacja ? w sqlite3).
- Gwarancja optymalnych zapytań z poprawnymi złączeniami JOIN.
- Zwracanie czytelnych słowników biznesowych zamiast surowych tabel.

Architektura „Domain Tools First” (standard bezpieczeństwa)



Zamiast dawać modelowi surowy SQL, programista tworzy **narzędzia domenowe**:

1. szukaj_filmow_gatunku(nazwa_gatunku: str)
2. pobierz_szczegoly_filmu(tytul: str)

Zalety:

- 100% odporność na SQL Injection (twarda parametryzacja ? w sqlite3).
- Gwarancja optymalnych zapytań z poprawnymi złączeniami JOIN.
- Zwracanie czytelnych słowników biznesowych zamiast surowych tabel.

Implementacja narzędzi domenowych w Pythonie

sages

Fragment modułu domain_tools.py:

```
def szukaj_filmow_gatunku(nazwa_gatunku: str) -> list[dict]:
    conn = sqlite3.connect("filmy.db")
    conn.row_factory = sqlite3.Row # Zwraca dict zamiast krotki!
    cursor = conn.cursor()
    cursor.execute(
        "SELECT tytul, ocena_filmweb, liczba_glosow FROM fakt_filmy f "
        "JOIN wymiar_gatunki g ON f.id_gatunku = g.id_gatunku "
        "WHERE LOWER(g.nazwa_gatunku) = LOWER(?)",
        (nazwa_gatunku,)
    )
    return [dict(r) for r in cursor.fetchall()]
```

sages

Demo

domain_tools.py

Ćwiczenie

cwiczenia/05_sql_tool_todo.py

Przerwa

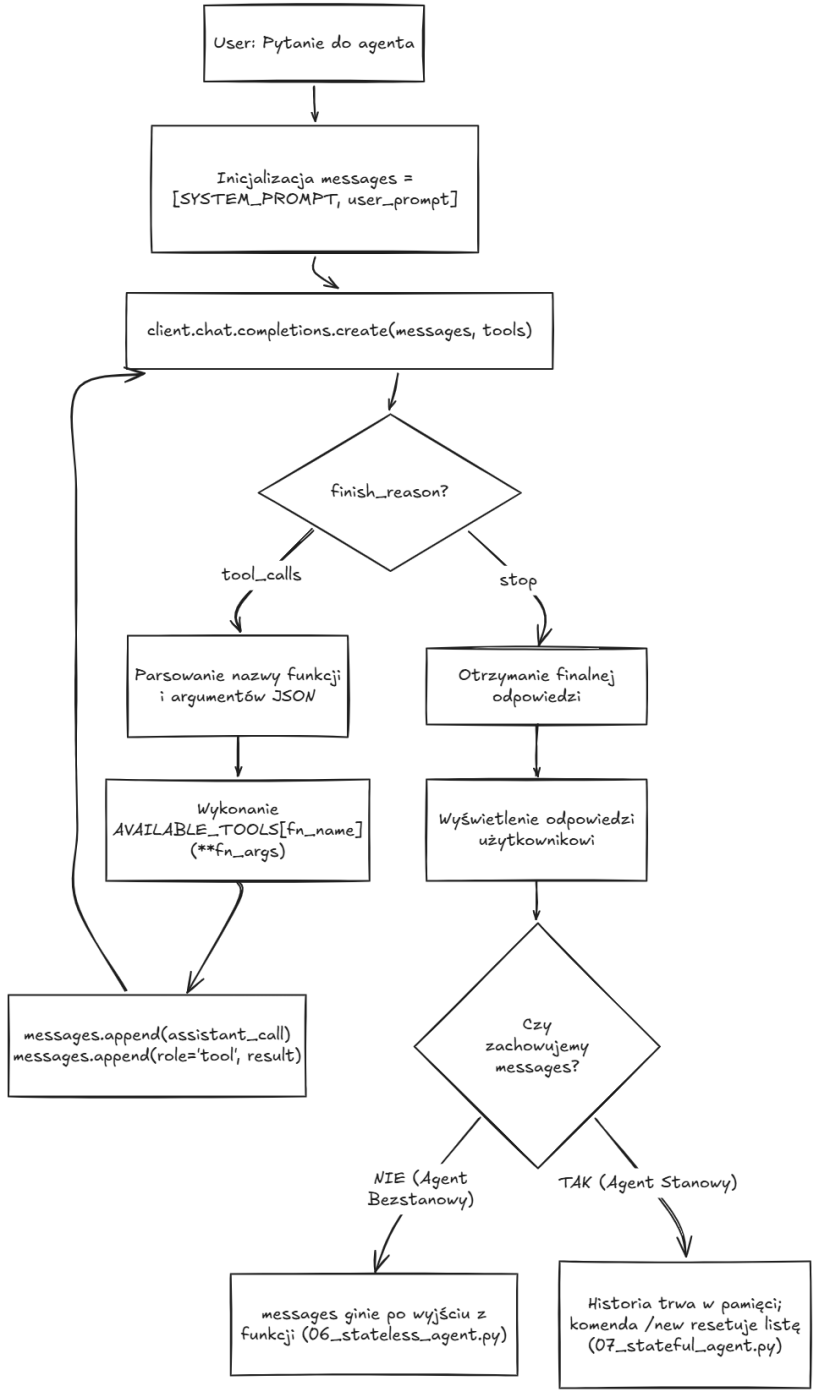
Pętla agentowa (Agentic Loop) i pamięć kontekstu

Pętla Agentowa (Agentic Loop):

- Pętla while True działająca dopóki model zwraca finish_reason == "tool_calls".
- Agent może wykonać wiele kroków narzędziowych w ramach jednego pytania!

Dwa kluczowe pytania o pamięć:

1. Czy model pamięta, co robił 5 sekund temu w poprzednim pytaniu?
2. Co się stanie, gdy po zwróceniu odpowiedzi wyczyścimy listę messages?



Ćwiczenie A: demonstracja braku pamięci

Eksperyment:

- Pytanie 1: *„Kto wyreżyserował film Diuna?”* → Odpowiedź: *Denis Villeneuve*.
- Pytanie 2: *„A kto w nim zagrał?”*

Wynik:

Model odpowiada: *„Nie wiem, o jaki film pytasz. Podaj tytuł.”*

Przyczyna:

Funkcja obsłużyła zadanie i zmienna lokalna messages została zniszczona. Model nie ma własnej pamięci!

Demo

06_stateless_agent.py

Ćwiczenie B: pętla stanowa i komenda /new

Rozwiązanie problemu pamięci:

- Zmienna conversation_history żyje przez cały czas działania bota.
- Każde nowe pytanie użytkownika i odpowiedź asystenta są dołączane do listy.

Implementacja polecenia `/new`:

```
if user_input.lower() == "/new":
    conversation_history = [SYSTEM_PROMPT]
    print("🧹 Pamięć wyczyszczona!")
    continue
```

Efekt: Pytanie: *"A kto w nim zagrał?"* działa bezbłędnie – model pamięta Incepcję!

sages

Demo

07_stateful_agent.py

Ćwiczenie

cwiczenia/06_chat_memory_todo.py

Problem „context stuffing” i limity okna

Zjawisko Context Stuffing:

Naiwne przesyłanie całej historii 50 wiadomości przy każdym pytaniu.

Skutki biznesowe i techniczne:

1. **Eksplozja kosztów:** Płacisz za wejściowe tokeny wszystkich poprzednich pytań wielokrotnie.
2. **Wzrost latencji:** Czas przetwarzania rośnie wraz z długością promptu.
3. **Efekt „Lost in the Middle”:** Modele mają tendencję do ignorowania instrukcji umieszczonych w środku bardzo długiego kontekstu.

Most do dnia 2: pierwszy krok w LangChain 1.0

sages

Co zrobiliśmy dzisiaj w czystym Pythonie?

- Zbudowaliśmy od zera silnik agenta: schematy JSON Schema, obsługę narzędzi, pętlę decyzyjną i pamięć (~150 linii kodu).

Jak wygląda to samo w LangChain 1.0?

```
from langchain.chat_models import init_chat_model
model = init_chat_model("gpt-4o-mini", temperature=0.1)
res = model.invoke("Czym różni się Chain od Agenta?")
print(res.content)
```

- Framework eliminuje powtarzalny boilerplate i wprowadza unifikację dostawców!

sages

Demo

08_langchain_intro.py

Podsumowanie dnia 1 i plan na dzień 2

sages

Podsumowanie Dnia 1 (Zrobione!):

- ✅ Środowisko uv, sekrety .env, surowe HTTP i oficjalne SDK openai.
- ✅ Streaming tokenów na żywo i modele lokalne w Ollamie (base_url).
- ✅ Własny silnik agenta z narzędziami plikowymi i bazą SQLite.
- ✅ Bezpieczne narzędzia domenowe i stanowa pamięć konwersacji (/new).

Dzień 2: Architektura Produkcyjna (09:00 – 16:30):

- 🚀 **LangSmith od rana:** Europejski serwer (EU) i tracing sesji (session_id).
- 🌐 **Agent ze światem:** Narzędzie sieciowe DuckDuckGo.
- ⚡ **FastAPI & Asyncio:** Mikroserwis bazy filmowej.
- 📚 **RAG:** Wyszukiwanie semantyczne na Wolnych Lekturach w Chroma.
- 🔌 **Standard MCP:** Własny serwer w FastMCP dla VSCode i agentów.

Ewolucja ekosystemu: LangChain 1.0

Koniec z rozproszeniem:

- Wcześniej: dziesiątki mikro-pakietów (langchain-core, langchain-community, langchain-experimental) i mylące importy.
- Teraz: jednolity import from langchain.chat_models import init_chat_model, from langchain.agents import create_agent.

Architektura Agent-Centric:

- Odejście od sztywnych łańcuchów sekwencyjnych na rzecz dynamicznych agentów decyzyjnych.

Nowoczesny standard inicjalizacji modeli:

```
model = init_chat_model("gpt-4o-mini", temperature=0.1)
# Zmiana na Claude lub Mistral to zmiana jednego stringa!
```

Wielu dostawców z init_chat_model() i pakiety partnerskie

sages

Uniwersalna fabryka modeli w LangChain 1.0:

Jeden spójny interfejs dla chmury i modeli lokalnych:

```
from langchain.chat_models import init_chat_model
```

Dostawcy, pakiety i zmienne środowiskowe:

Dostawca	Model (model lub prefiks)	Wymagana paczka Pythona	Zmienna środowiskowa
OpenAI	"openai:gpt-4o-mini"	langchain-openai	OPENAI_API_KEY
Anthropic	"anthropic:claude-3-5-sonnet-latest"	langchain-anthropic	ANTHROPIC_API_KEY
Google AI Studio	"google_genai:gemini-1.5-flash"	langchain-google-genai	GEMINI_API_KEY
Google Vertex AI	"google_vertexai:gemini-1.5-pro"	langchain-google-vertexai	GOOGLE_APPLICATION_CREDENTIALS
Groq (LPU)	"groq:llama-3.3-70b-versatile"	langchain-groq	GROQ_API_KEY
DeepSeek	"deepseek:deepseek-chat"	langchain-deepseek	DEEPSEEK_API_KEY
Ollama (Lokalna)	"ollama:llama3.2"	langchain-ollama	*(brak – port lokalny)*

Zasada modularności: Brak wielkiego monolitu — instalujesz wyłącznie pakiet tego dostawcy, z którego faktycznie korzystasz!

Lokalne LLM i on-premise: Ollama i własne endpointy

sages

Prywatność i On-Premise (Zero danych w chmurze publicznej):

Kluczowe dla bankowości, telekomunikacji, analizy poufnych logów sieciowych i konfiguracji urządzeń.

Integracja z Ollamą (langchain-ollama):

```
# 1. Domyślna instancja lokalna (localhost:11434)
model_local = init_chat_model("ollama:llama3.2", temperature=0.1)

# 2. Zewnętrzny serwer z GPU w sieci firmowej
model_server = init_chat_model(
    "ollama:qwen2.5-coder:32b",
    base_url="http://10.200.4.15:11434",
    temperature=0.0
)
```

Ollama i silniki open-source jako endpointy kompatybilne z OpenAI:

```
# Dowolny silnik on-premise (Ollama, vLLM, llama.cpp, TGI)
model_vllm = init_chat_model(
    "openai:mistralai/Mistral-Small-24B",
    base_url="http://vllm.internal.corp:8000/v1",
    api_key="brak-wymagan"
)
```

Koncepcja MRKL (Modular Reasoning, Knowledge and Language)

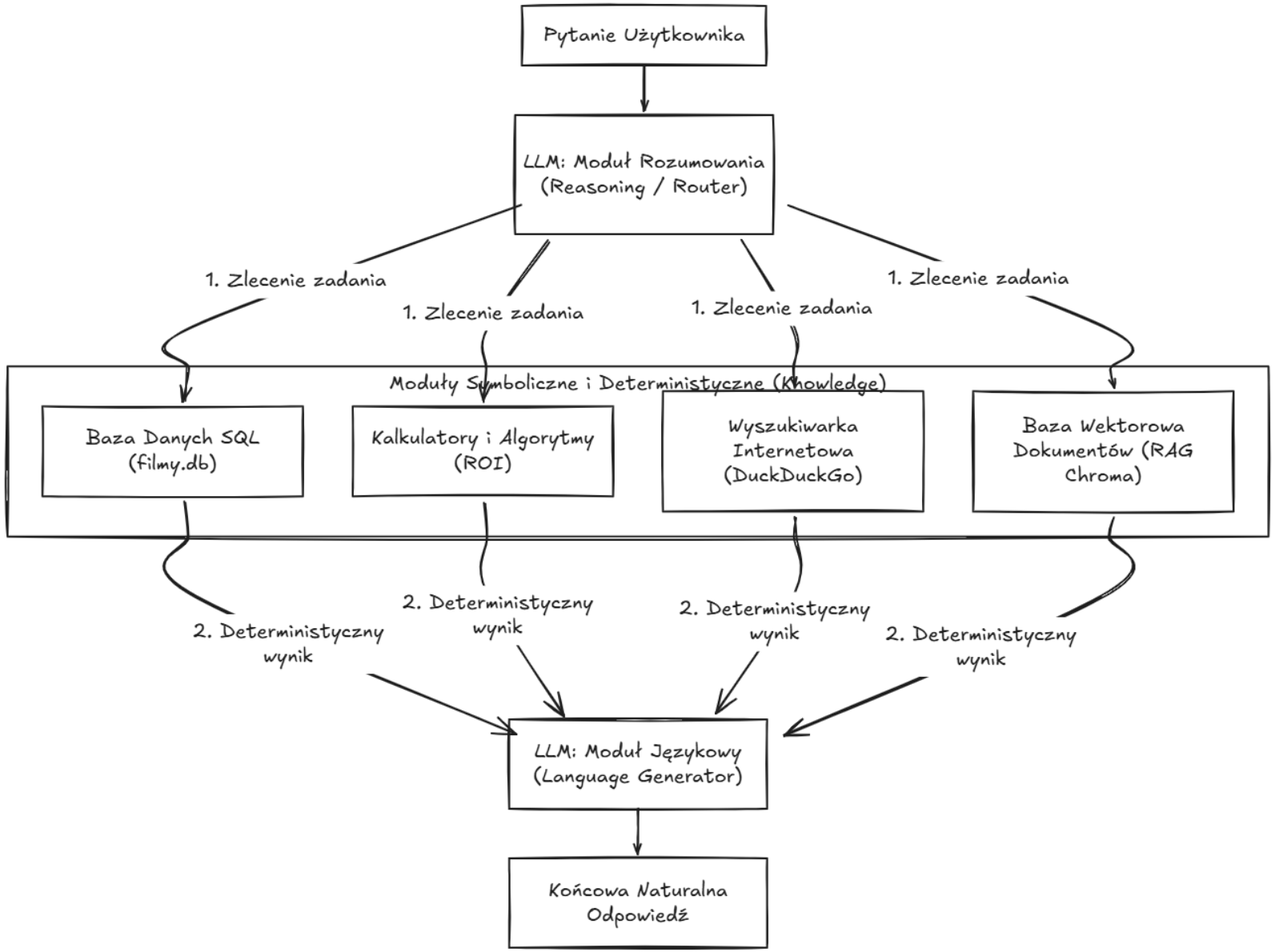
Paper naukowy (AI21 Labs, 2022): *"MRKL Systems: A modular, neuro-symbolic architecture..."*

Trzy filary architektury:

1. **Reasoning (Rozumowanie):** LLM analizuje pytanie użytkownika, rozbija problem i wybiera narzędzia.
2. **Knowledge (Wiedza Symboliczna):** Deterministyczne moduły: baza SQLite, API, kalkulator, wyszukiwarka internetowa.
3. **Language (Synteza Językowa):** LLM formułuje płynną odpowiedź na podstawie twardych danych od modułów.

Wniosek: MRKL to formalny rodowód współczesnego *Tool Calling* i *ReAct*.

Koncepcja MRKL (Modular Reasoning, Knowledge and Language)



Streaming odpowiedzi w LangChain model.stream()

sages

Strumieniowanie tokenów w standardzie LangChain:

```
model = init_chat_model("gpt-4o-mini", temperature=0.2)
for chunk in model.stream("Wyjaśnij pojęcie MRKL w 2 zdaniach:"):
    print(chunk.content, end="", flush=True)
```

Zaleta architektoniczna:

Ten sam interfejs generatora działa identycznie dla OpenAI, Claude, modeli z HuggingFace czy Ollamy.

sages

Demo

09_langchain_streaming.py

Dlaczego LangSmith włączamy od zaraz?

Problem produkcyjny LLM: Kod kończy się statusem HTTP 200, ale model odpowiedział bzdurę (halucynacja).

- Tradycyjny logger (logging.info) nie pokazuje:
- Pełnego promptu wysłanego do API z wstrzykniętą historią.
- Kosztu tokenów każdego pośredniego kroku agenta.
- Latencji poszczególnych narzędzi w pętli.

Zasada: Włączamy **LangSmith** teraz i przez resztę dnia obserwujemy w przeglądarce graf każdego kolejnego ćwiczenia!

Konfiguracja LangSmith EU (RODO / GDPR)

- **Domyślnie:** Serwery w USA (api.smith.langchain.com).
- **Dla Europy (EU Data Residency):** Dedykowany endpoint zgodny z regulacjami UE:

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=lsv2_pt_twoj_klucz
LANGCHAIN_PROJECT=nazwa_twojego_projektu
LANGCHAIN_ENDPOINT="https://eu.api.smith.langchain.com"
```

- **Zero zmian w kodzie:** Framework sam wykrywa zmienne środowiskowe i włącza telemetrię!

Identyfikacja sesji i użytkownika (session_id)

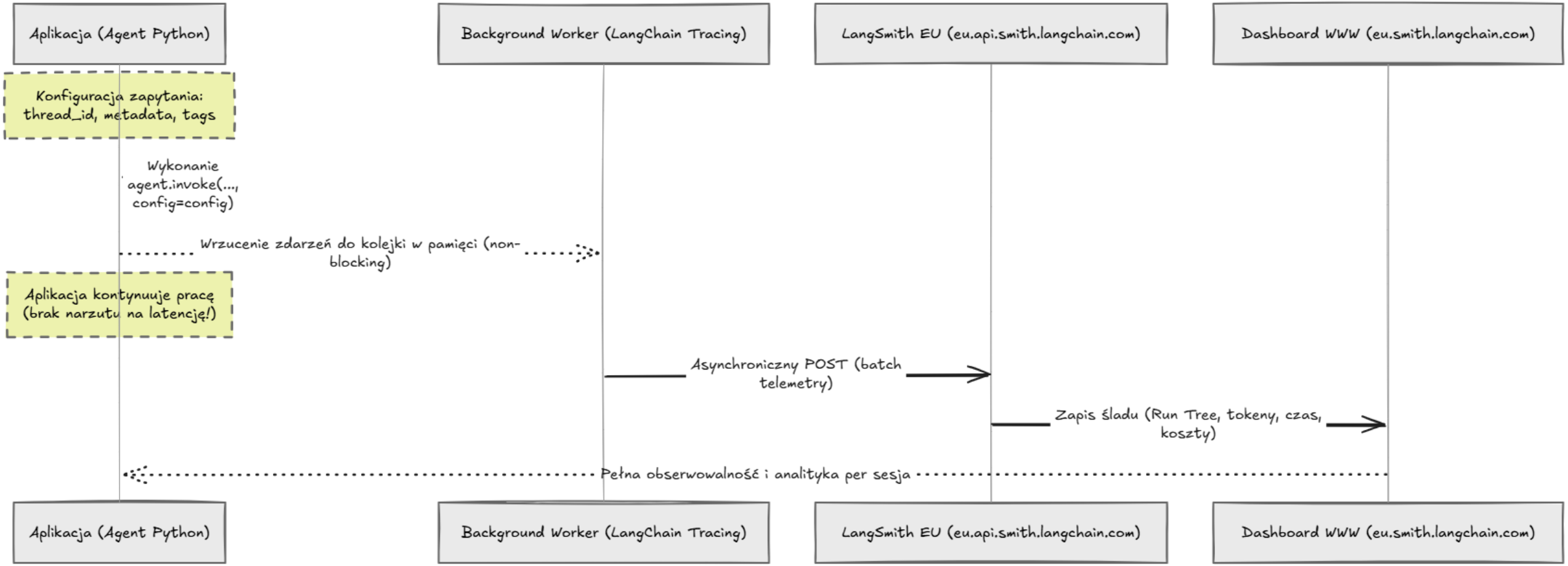
W systemie z wieloma użytkownikami musimy powiązać ślady (Traces) z konkretną rozmową:

```
import uuid
from datetime import datetime

user_login = "jan.kowalski"
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
session_id = f"{user_login}_{timestamp}_{uuid.uuid4().hex[:6]}"

config = {
    "configurable": {"thread_id": session_id},
    "metadata": {"session_id": session_id, "user": user_login},
    "tags": ["warsztat", "bol-chat-bot", user_login]
}
agent.invoke(..., config=config)
```

Identyfikacja sesji i użytkownika (session_id)



Inspekcja śladu w panelu LangSmith EU

Co widzimy w dashboardzie eu.smith.langchain.com ?

- Drzewo wywołań (DAG): zapytanie → model → narzędzie → model.
- Dokładny czas w milisekundach na każdym etapie.
- Liczbę tokenów wejściowych i wyjściowych.
- Obliczony koszt zapytania w USD.
- Surowy JSON wysłany i odebrany z OpenAI.

sages

Demo

10_langsmith_session.py

sages

Ćwiczenie

cwiczenia/07_langsmith_todo.py

Przerwa

Agent LangChain: dekorator @tool

Koniec z ręcznym pisaniem słowników JSON Schema:

```
from langchain.tools import tool

@tool
def szukaj_filmow(gatunek: str) -> list[dict]:
    """Wyszukuje filmy w bazie SQLite po gatunku (np. Sci-Fi)."""
    # Czysty kod w Pythonie...
```

LangChain automatycznie:

- Odczytuje docstring funkcji jako description narzędzia dla LLM.
- Odczytuje type hinty (str, int, list) jako typy w schemacie JSON.
- Waliduje argumenty wejściowe przed uruchomieniem kodu!

Pamięć wątków z InMemorySaver()

sages

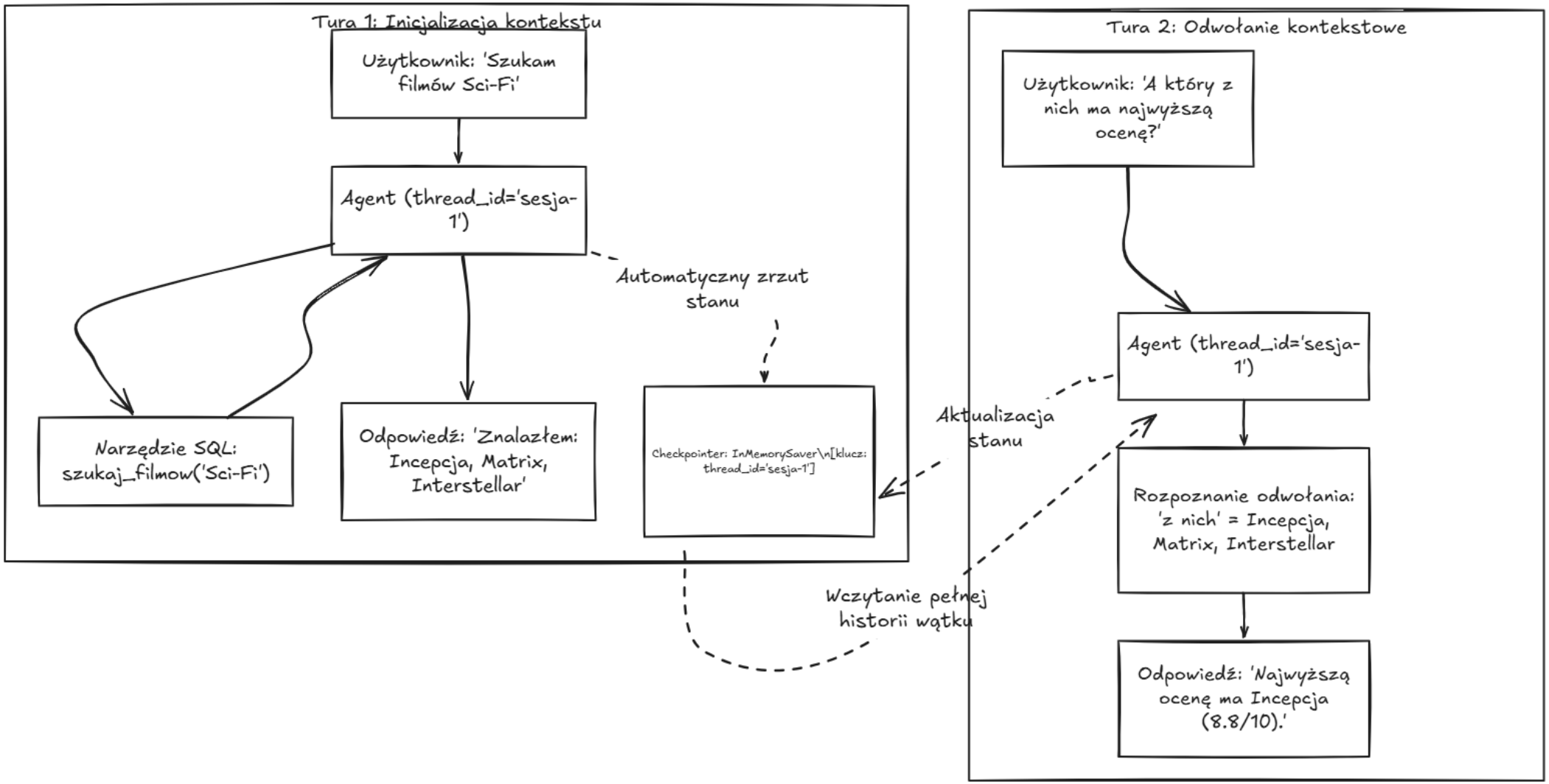
Nowoczesne zarządzanie stanem w LangChain 1.0:

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent

checkpointer = InMemorySaver()
agent = create_agent(
    model=model,
    tools=[...],
    checkpointer=checkpointer
)
```

Pamięć działa automatycznie pod kluczem thread_id w konfiguracji zapytania!

Pamięć wątków z InMemorySaver



sages

Demo

11_langchain_agent_tools.py

Po co agentowi dostęp do internetu?

Ograniczenia systemów zamkniętych:

- Baza SQLite zna tylko filmy z lat 2002–2024.
- Model ma zamrożoną wiedzę (*Knowledge Cutoff*).

Nowe wyzwanie biznesowe:

- Pytanie użytkownika: *"Jakie filmy wchodzą do kin w ten piątek?"*

Rozwiązanie:

- Narzędzie sieciowe (**DuckDuckGo Search** / **Tavily**), które przeszukuje otwarty Internet w czasie rzeczywistym.

Narzędzie sieciowe w Pythonie

sages

Implementacja darmowego wyszukiwania DuckDuckGo:

```
from ddgs import DDGS
from langchain.tools import tool

@tool
def szukaj_w_internecie(zapytanie: str) -> str:
    """Wyszukuje bieżące informacje i premiery kinowe w Internecie."""
    with DDGS() as ddgs:
        wyniki = list(ddgs.text(zapytanie, max_results=3))
        return "\n\n".join([f"{r['title']}: {r['body']}" for r in wyniki])
```

Demo

12_agent_internet.py

Wzorzec Autonomous Tool Routing

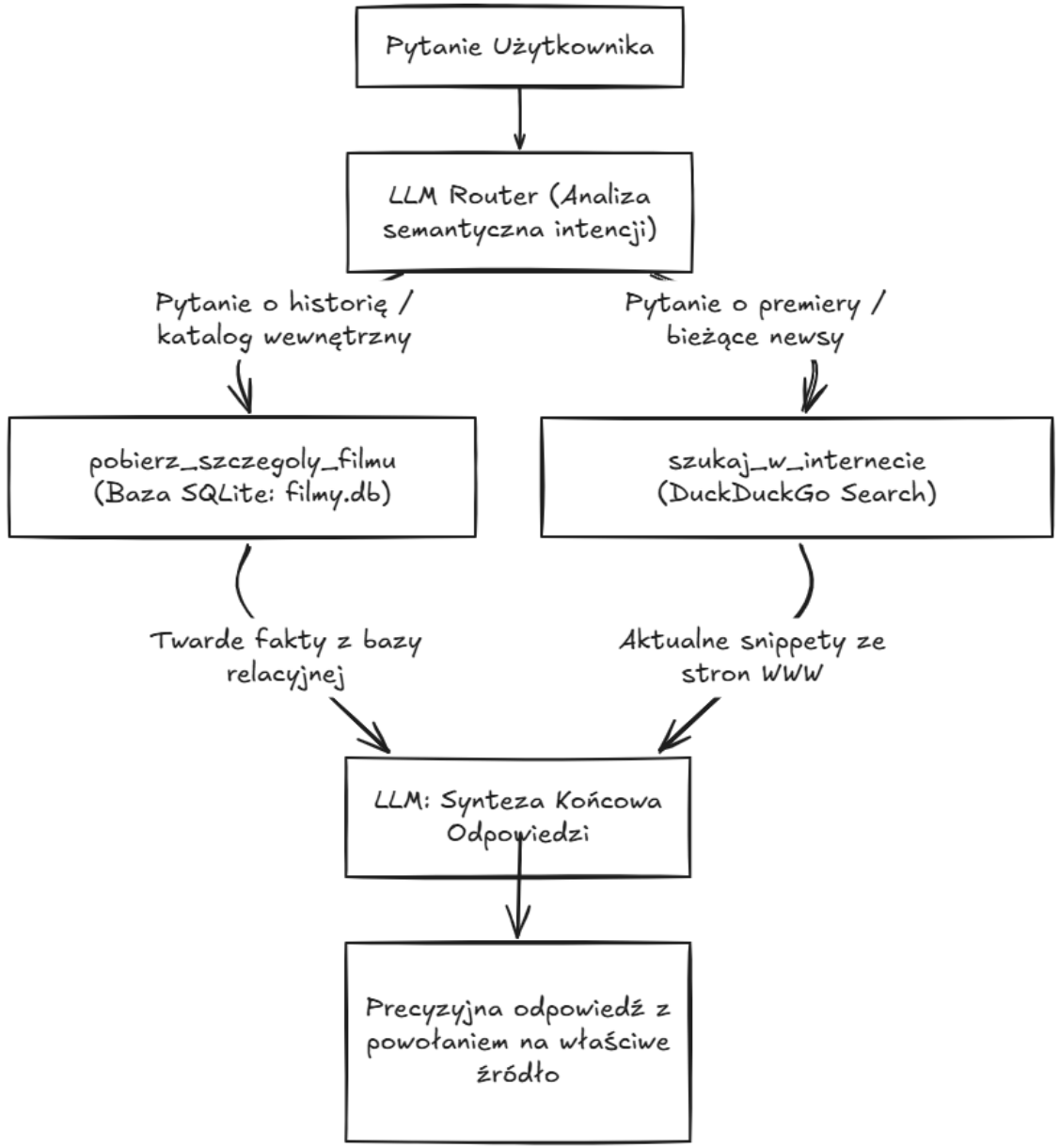
Jak model decyduje, skąd wziąć wiedzę?

- Pytanie 1: *"Kto wyreżyserował Incepcję?„*
  → Model analizuje narzędzia i wybiera pobierz_szczegoly_filmu (baza SQLite).

- Pytanie 2: *"Co nowego grają w kinach w tym tygodniu?„*
  → Model wie, że tego nie ma w bazie, i wybiera szukaj_w_internecie!

Zero instrukcji if/else w Pythonie:

- Logika routingu wynika z semantycznego rozumienia promptu przez LLM.



Podgląd routingu w LangSmith EU

Inspekcja w panelu webowym:

- Pytanie 1: W grafie widać wywołanie pobierz_szczegoly_filmu (czas: 12 ms).
- Pytanie 2: W grafie widać wywołanie szukaj_w_internecie (czas: 840 ms).

Pełna przejrzystość działania systemu przed przerwą obiadową!

sages

Ćwiczenie

cwiczenia/08_web_tool_todo.py

Przerwa

Architektura korporacyjna: separacja warstw

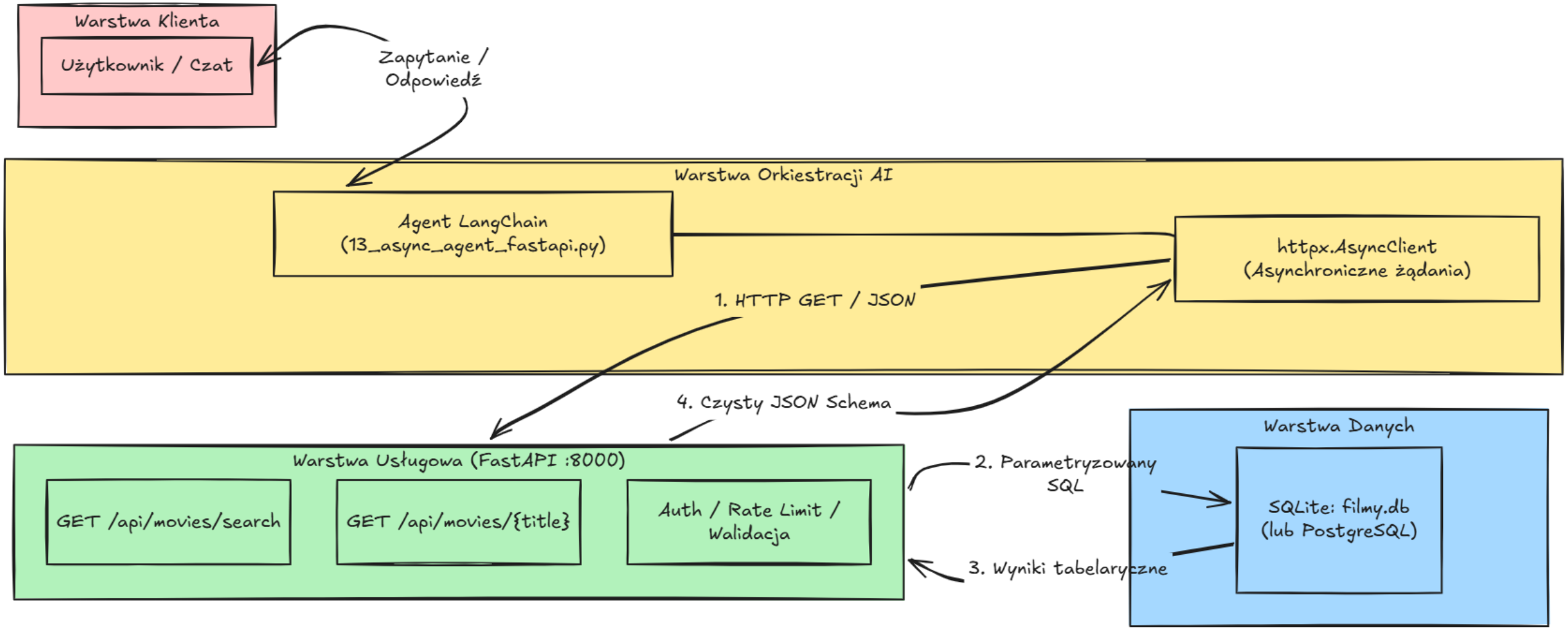
Dlaczego agent NIE powinien bezpośrednio łączyć się z bazą danych?

1. **Zasada najmniejszych uprawnień (Least Privilege):**
   Agent nie powinien mieć dostępu do sterownika bazy.
2. **Skalowanie:**
   Backend bazodanowy skaluje się inaczej niż warstwa wnioskowania AI.
3. **Współdzielenie:**
   Z tego samego REST API korzysta aplikacja webowa, aplikacja mobilna oraz nasz agent AI.

Architektura trójwarstwowa:

1. Agent (LangChain)
2. API (FastAPI)
3. Baza Danych

Architektura Korporacyjna: Separacja Warstw



Mikroserwis w FastAPI

sages

Definicja endpointów w FastAPI:

- GET /api/movies/search?genre=Sci-Fi
- GET /api/movies/{title}

Automatyczna dokumentacja: Swagger UI pod adresem http://localhost:8000/docs

Uruchomienie:

```
uv run uvicorn kino_api:app --port 8000 --reload
```

sages

Demo

kino_api.py

Asynchroniczność w AI (asyncio i httpx)

Problem synchronicznego requests.get():

- Blokuje cały wątek roboczy serwera na czas oczekiwania na odpowiedź sieciową.

Standard asynchroniczny:

- Biblioteka **httpx.AsyncClient** zamiast requests.
- Asynchroniczne wywołanie agenta:

```
odpowiedz = await agent.ainvoke({"messages": [...]})
```

Korzyść:

- Jeden proces backendowy może równolegle obsługiwać dziesiątki zapytań do modeli LLM bez blokowania pętli zdarzeń!

Agent LangChain konsumujący FastAPI

sages

Asynchroniczne narzędzie odpytujące mikroserwis:

```
@tool
async def api_szukaj_filmow(gatunek: str) -> dict:
    """Wyszukuje filmy w API kina na podstawie gatunku."""
    async with httpx.AsyncClient() as client:
        res = await client.get(f"{API_URL}/api/movies/search", params={"genre": gatunek})
        return res.json()
```

W konsoli Uvicorna widzimy zapytania HTTP generowane automatycznie przez agenta!

W LangSmith widzimy dokładny czas żądania HTTP.

sages

Demo

13_async_agent_fastapi.py

Ćwiczenie

cwiczenia/09_async_fastapi_todo.py

Przerwa

Problem „gadatliwego tekstu" w backendzie

Co model zwraca domyślnie?

- *"Oczywiście! Z przyjemnością przedstawiam rekomendację filmów Sci-Fi: 1. Incepcja..."*

Dlaczego to problem dla programisty?

- Jak zapisać to w relacyjnej bazie danych?
- Jak przekazać to do frontendu w Reactcie lub aplikacji mobilnej?
- Pisanie wyrażeń regularnych (regex) to gwarancja awarii przy drobnej zmianie formatowania przez model.

Rozwiązanie: Wymuszenie **Structured Output** (kontrolowane dekodowanie do JSON Schema).

Ustrukturyzowane wyjścia z Pydantic with_structured_output()

Definicja modelu danych w Pydantic:

```
from pydantic import BaseModel, Field

class FilmRekomendacja(BaseModel):
    tytul: str = Field(description="Tytuł filmu")
    ocena: float = Field(description="Ocena 1-10")
    uzasadnienie: str = Field(description="Dlaczego warto obejrzeć")

class RaportKinowy(BaseModel):
    profil_widza: str = Field(description="Profil odbiorcy")
    propozycje: list[FilmRekomendacja]
    podsumowanie: str
```

Wymuszenie w modelu:

```
structured_model = model.with_structured_output(RaportKinowy)
raport: RaportKinowy = structured_model.invoke(prompt)
```



sages

Demo

14_structured_output.py

sages

Ćwiczenie

cwiczenia/10_pydantic_todo.py

Embeddings i wyszukiwanie Semantyczne

sages

Słowa kluczowe vs Semantyka:

- Wyszukiwanie tradycyjne (SQL LIKE): szuka identycznych liter (*"lekarz" <> "doktor"*).
- Wyszukiwanie wektorowe: szuka **znaczenia** tekstu w przestrzeni wielowymiarowej.

Modele Embeddingowe (np. `text-embedding-3-small`):

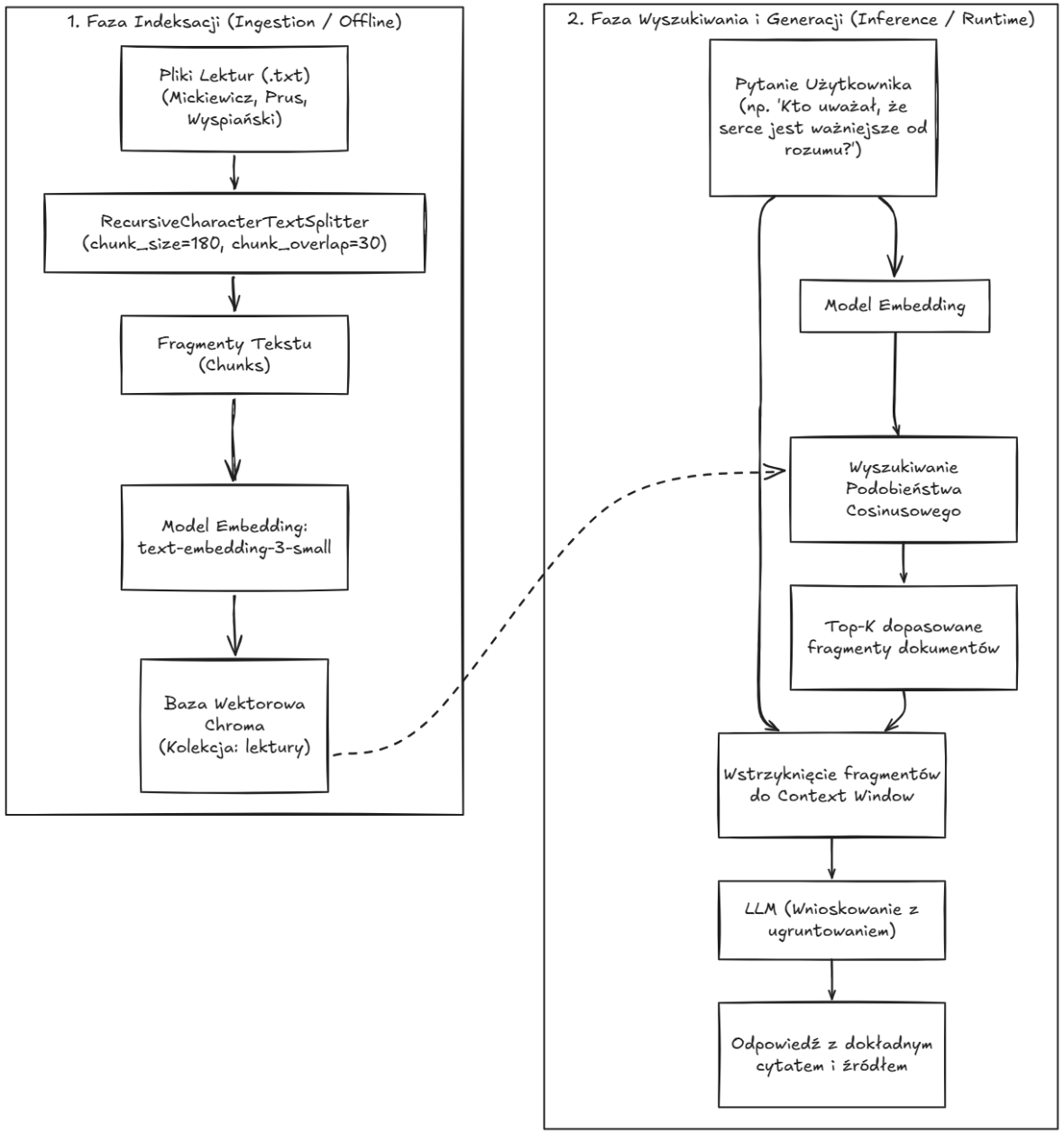
- Zamieniają tekst na wektor 1536 liczb rzeczywistych.
- **Podobieństwo Cosinusowe:**
  - Mierzy kąt między wektorami.
  - Kąt bliski zeru = teksty o identycznym znaczeniu.

Pipeline RAG na Wolnych Lekturach

sages

Architektura RAG (Retrieval-Augmented Generation):

1. **Dokumenty:**
   Teksty z Wolnych Lektur (Mickiewicz, Prus, Wyspiański).
2. **Chunking:**
   Podział na fragmenty (RecursiveCharacterTextSplitter(chunk_size=180, chunk_overlap=30)).
3. **Wektoryzacja:**
   Zamiana fragmentów na wektory i zapis w bazie **Chroma**.
4. **Retriever:**
   Wyciągnięcie Top-K fragmentów najbardziej pasujących do pytania użytkownika.
5. **Generacja:** Wstrzyknięcie cytatów do promptu modelu.



Baza wektorowa jako narzędzie agenta

Przekształcenie bazy wektorowej w narzędzie (`create_retriever_tool`):

```
tool_literacki = create_retriever_tool(
    retriever=retriever,
    name="szukaj_w_literaturze",
    description="Wyszukuje motywy filozoficzne i cytaty z dzieł Mickiewicza, Prusa i Wyspiańskiego."
)
agent = create_agent(model=model, tools=[tool_literacki, ...])
```

Efekt: Agent sam decyduje, kiedy sięgnąć po cytaty z literatury!

Demo

15_rag_lektury.py

Inspekcja cytatów RAG w LangSmith EU

Weryfikacja wierności (Faithfulness):

- Otwieramy LangSmith EU pod adresem eu.smith.langchain.com.
- W drzewie wywołań klikamy w węzeł Retriever.
- **Widzimy dokładnie, jakie fragmenty tekstu Chroma zwróciła modelowi!**

Klucz do audytu: Możemy natychmiast zweryfikować, czy model zacytował prawdę, czy coś zmyślił.

Demo (cd)

15_rag_lektury.py

Ćwiczenie

cwiczenia/11_rag_tool_todo.py

Przerwa

Rewolucja MCP: koniec fragmentacji narzędzi

Problem integracji w świecie AI:

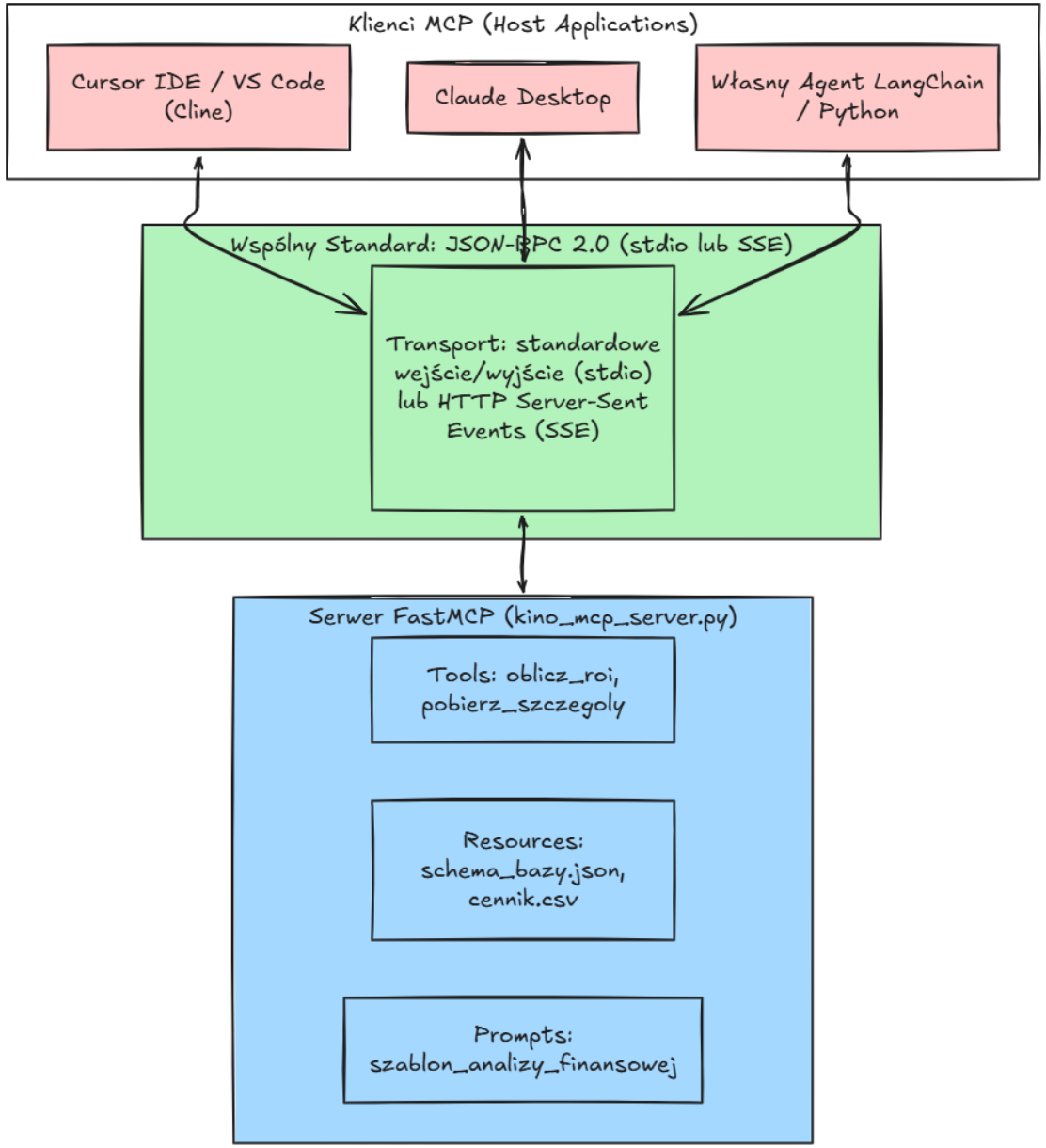
- Każda platforma (LangChain, Cursor, Claude Desktop, Copilot) wymagała własnego formatu narzędzi.

Rozwiązanie Anthropic (listopad 2024):

- **Model Context Protocol (MCP)**.

Zasada działania:

- Otwarty standard komunikacji oparty o protokół JSON-RPC 2.0.
- Piszesz serwer narzędzi **jeden raz**, a podłączasz go do każdego edytora i każdego agenta na świecie!



Tryby transportu MCP: stdio vs SSE

Cecha	Tryb stdio (Standard I/O)	Tryb SSE (Server-Sent Events HTTP)
Mechanizm	Komunikacja przez strumienie procesu (stdin/stdout). Klient sam uruchamia proces serwera.	Komunikacja sieciowa przez protokół HTTP. Serwer działa jako usługa w sieci.
Zastosowanie	Narzędzia lokalne, integracje z VSCode, Cursorem, skryptami na dysku.	Zdalne mikroserwisy, chmura, klastry Docker/Kubernetes.
Bezpieczeństwo	Maksymalne: zero otwartych portów sieciowych, brak nasłuchu w sieci.	Wymaga autoryzacji tokenem Bearer i szyfrowania HTTPS/TLS.

Własny serwer w Pythonie: FastMCP

sages

Oficjalna biblioteka `FastMCP`:

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Kino-Analytics-Server")

@mcp.tool()
def oblicz_roi(przychody_pln: float, koszty_pln: float) -> str:
    """Oblicza ROI oraz zysk netto filmu w PLN."""
    zysk = przychody_pln - koszty_pln
    roi = (zysk / koszty_pln) * 100
    return f"Zysk: {zysk:,.2f} PLN | ROI: {roi:.2f}%"

if __name__ == "__main__":
    mcp.run() # Domyślny transport: stdio
```

Demo

kino_mcp_server.py

Ćwiczenie

cwiczenia/12_fastmcp_todo.py

Podłączenie MCP do VSCode i Cursora

Konfiguracja edytora (np. `cline_mcp_settings.json` lub Cursor MCP):

```
{
  "mcpServers": {
    "kino-server": {
      "command": "uv",
      "args": ["run", "--directory", "/sciezka/do/projektu", "kino_mcp_server.py"]
    }
  }
}
```

Efekt w czacie edytora:

Wpisujesz: *"Sprawdź w bazie kina zysk z Incepcji i oblicz ROI dla kosztów 200 mln i przychodów 830 mln."*

Model w edytorze sam odpala Twój lokalny serwer w Pythonie!

Ewaluacja jakości na produkcji: LLM-as-a-Judge

sages

Jak testować systemy AI po wdrożeniu?

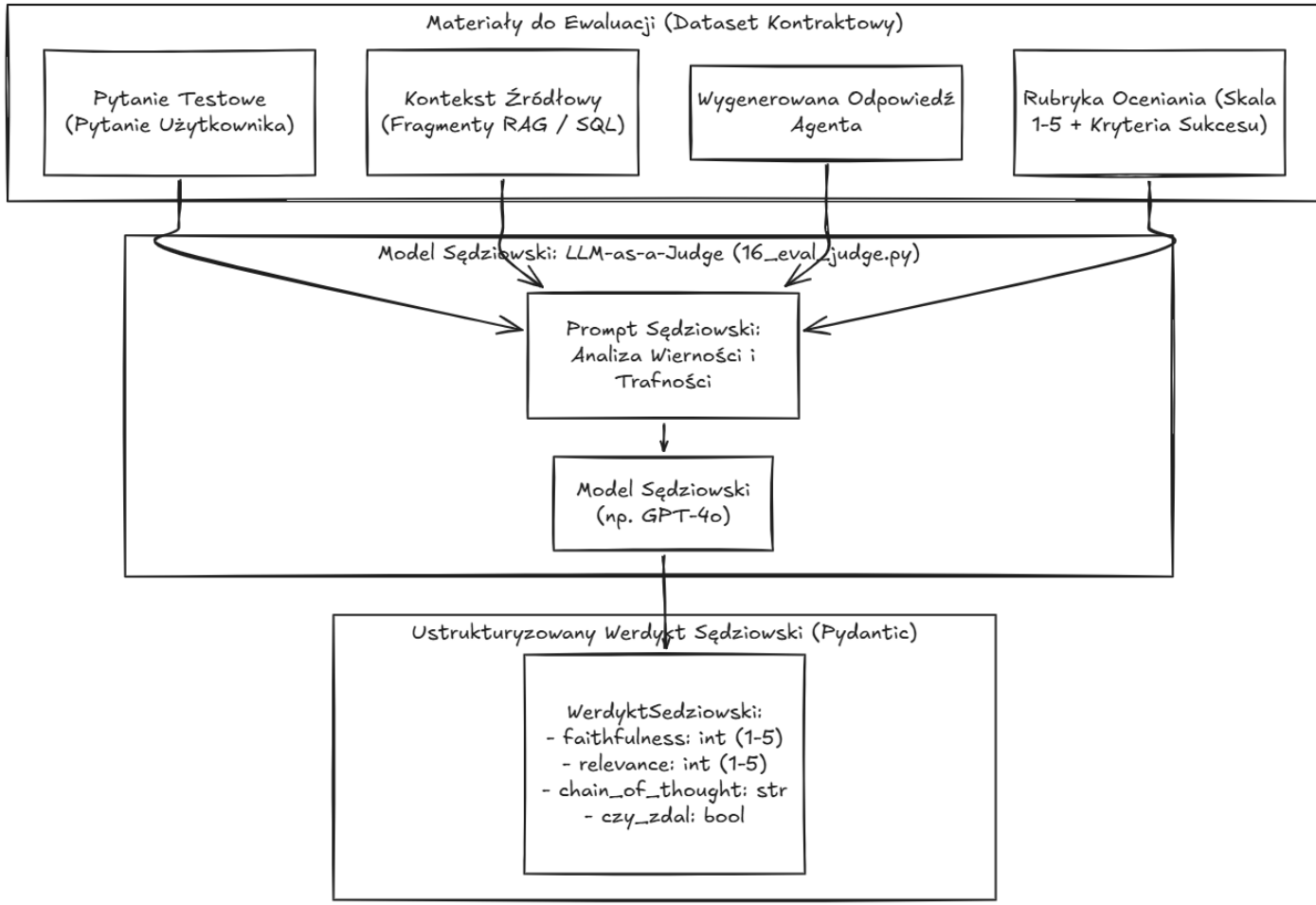
Tradycyjne testy asercji (assert response == "...") zawodzą z powodu stochastyczności tekstu.

Złoty standard: LLM-as-a-Judge

Użycie silniejszego modelu (np. GPT-4o) z promptem sędziowskim do oceny odpowiedzi naszego bota na zbiorze referencyjnym (*Golden Dataset*).

Kluczowe metryki ewaluacyjne:

1. **Faithfulness (Wierność):**
   Czy odpowiedź opiera się wyłącznie na danych z bazy i cytatach RAG?
2. **Answer Relevance (Trafność):**
   Czy model odpowiedział ściśle na pytanie użytkownika?



Demo

16_eval_judge.py

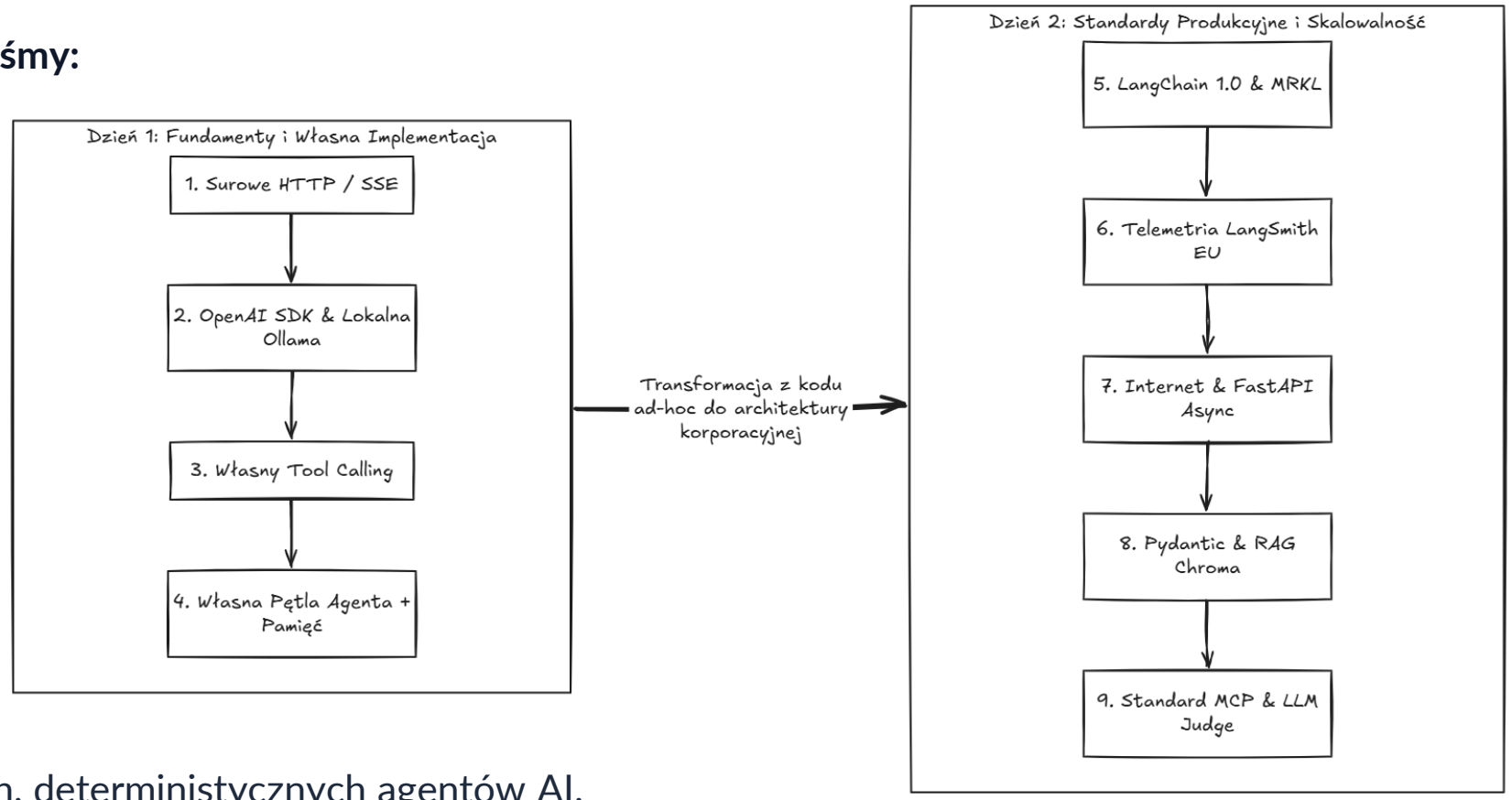
Ankieta

Zeskanuj kod i wypełnij ankietę po szkoleniu.

Link: https://sages.link/155586

Podsumowanie całego szkolenia

Pełna ścieżka inżynierska, którą przeszliśmy:



Co zyskaliście jako inżynierowie?

- Zdolność do budowania bezpiecznych, deterministycznych agentów AI.
- Zrozumienie kosztów, tokenów i pamięci pod maską.
- Umiejętność wdrażania nowoczesnych standardów rynkowych (LangChain, LangSmith, MCP).